

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
KHOA ĐIỆN TỬ VIỄN THÔNG**



BÁO CÁO ĐỒ ÁN 2

Sinh viên thực hiện:

Họ và tên:

Mã sinh viên:

Lớp khóa học:

Nguyễn Huyền Trang

20020727

QH-2020-I/CQ-R

Giảng viên hướng dẫn: PGS. TS. Dương Xuân Biên

Hà Nội, 2023.

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
KHOA ĐIỆN TỬ VIỄN THÔNG**



BÁO CÁO ĐỒ ÁN 2

Sinh viên thực hiện:

Họ và tên:

Nguyễn Huyền Trang

Mã sinh viên:

20020727

Lớp khóa học:

QH-2020-I/CQ-R

Giảng viên hướng dẫn: PGS. TS Dương Xuân Biên

Hà Nội, 2023.

CHƯƠNG 1. TỔNG QUAN VỀ VẤN ĐỀ NGHIÊN CỨU	5
1.1. Hệ thống đa robot.	5
1.1.1. Giới thiệu và phân loại các hệ thống đa robot.	5
1.1.2. Những thách thức trong lĩnh vực đa robot.	5
1.2. Tổng quan về các bài toán trong hệ đa robot	6
1.3. Đặt vấn đề	7
1.4. Tổng quan nội dung đồ án.	8
CHƯƠNG 2. MÔ HÌNH HÓA CÁNH TAY ROBOT	9
2.1. Mô hình hóa động học cho cánh tay robot.	9
2.1.1. Phân tích mô hình cánh tay 6 bậc tự do	9
2.1.2. Đặt các hệ tọa độ cho mô hình.	10
2.2.3. Xây dựng bảng DH	11
2.2.4. Tính các ma trận H.	13
2.2.5. Tính các ma trận D.	15
2.2.6. Kiểm tra hệ tọa độ điểm thao tác.	15
2.2. Mô hình hóa động lực học cho cánh tay robot.	18
2.2.1. Ma trận cosin chỉ hướng của các khâu theo hệ tọa độ cố định	18
2.2.2. Tọa độ điểm cuối các khâu theo hệ tọa độ cố định	18
2.2.3. Tính tọa độ trọng tâm các khâu theo hệ tọa độ địa phương	18
2.2.4. Tính tọa độ trọng tâm các khâu theo hệ tọa độ cố định	19
2.2.5. Ma trận khối lượng M.	19
2.2.6. Ma trận Coriolis C.	20
2.2.7. Ma trận thế năng trọng trường G	20
2.3. Mô hình hóa động lực học trong không gian thao tác.	20
CHƯƠNG 3. ĐIỀU KHIỂN TẬP TRUNG	22
3.1. Cơ sở lý thuyết	22
3.2. Xác định bài toán.	22
3.3. Mô phỏng hoạt động cánh tay robot.	23
3.3.1. Thiết lập mô hình robot.	23
3.3.2. Xây dựng môi trường làm việc.	24
3.3.3. Lập quỹ đạo chuyển động cho robot.	27
3.3.4. Kết quả thực hiện.	27

CHƯƠNG 4. LẬP KẾ HOẠCH CHUYỂN ĐỘNG CHO HỆ THỐNG ĐA CÁNH TAY ROBOT BẰNG PHƯƠNG PHÁP ĐIỀU KHIỂN PHÂN TÁN.	33
4.1. Tiếp cận vấn đề.	33
4.1.1. Thách thức trong điều khiển hệ đa robot	33
4.1.2. Một số nghiên cứu liên quan.	34
4.1.2. Xác định vấn đề.	35
4.2. Học tăng cường	36
4.2.1. Khái quát về học tăng cường	36
4.2.2. Học Tăng cường Đa tác tử.	37
4.3. Phương pháp Soft Actor-Critic	38
4.4. Phương án thực hiện.	39
4.4.1. Học tăng cường phân tán với mô phạm mẫu từ chuyên gia.	39
4.4.2. Khởi tạo nhiệm vụ vận hành cho chuyển động của hệ đa cánh tay.	39
4.4.3. Công thức huấn luyện đa tác tử trong hệ học tăng cường.	40
4.4.4. Huấn luyện lại model tại những nhiệm vụ không thực hiện được.	40
CHƯƠNG 5. KẾT QUẢ THỰC HIỆN	41
5.1. Phát biểu bài toán	41
5.2. Tiêu chí đánh giá.	42
5.3. Kết quả và đánh giá.	43
5.3.1. Thời gian huấn luyện mô hình.	43
5.3.2. Tỷ lệ thành công khi tăng số lượng tác tử.	44
5.3.3. Hiệu quả hoạt động của mô hình huấn luyện.	45
5.4. Kết luận.	46
	46
TÀI LIỆU THAM KHẢO	46

CHƯƠNG 1. TỔNG QUAN VỀ VẤN ĐỀ NGHIÊN CỨU

1.1. Hệ thống đa robot.

1.1.1. Giới thiệu và phân loại các hệ thống đa robot.

Việc sử dụng cánh tay robot đã trở nên phổ biến trong xã hội công nghệ ngày nay. Những cỗ máy này hoàn thành các nhiệm vụ từ tầm thường đến kỳ diệu mà hầu hết mọi người không quan tâm đến cách chúng tính toán chuyển động của chúng, xét cho cùng thì việc điều khiển chuyển động của cánh tay của một người phức tạp hơn nhiều so với một cánh tay robot đơn giản nhưng nó được thực hiện với rất ít nỗ lực. Thực tế là mô hình toán học cho chuyển động của robot thường phi tuyến tính.

Lập quỹ đạo chuyển động tối ưu là phương tiện chính để đạt được sự kiểm soát hiệu quả các bộ điều khiển robot trong không gian vật lý. Các phương pháp đạt được điều khiển tối ưu đã được nghiên cứu rộng rãi trong nửa thế kỷ qua, nhưng chỉ với sự gia tăng sức mạnh tính toán và các kỹ thuật số tiên tiến trong khoảng thập kỷ qua thì điều khiển tối ưu giờ đây mới có thể được thực hiện trong nhiều nhiệm vụ khác nhau. Lĩnh vực robot trong lịch sử đã sử dụng một số hình thức tối ưu hóa để đạt được hiệu quả trong chuyển động của các hệ thống với mục tiêu vận hành tự động hoàn thành nhiệm vụ mong muốn với ít sự tham gia điều khiển của con người nhất có thể.

Theo lẽ thường, từ thế kỷ này sang thế kỷ khác, trong khi lao động cực nhọc, con người luôn cần sự hỗ trợ của các công cụ lao động để được giảm bớt sức lao động. Rồi từ công cụ giản đơn, thô sơ, đến máy móc ngày càng tinh vi, hiện đại, con người càng tiến theo chiều hướng dựa vào những thứ giúp mình đỡ nặng nhọc hơn, làm được nhiều việc trong cùng một thời gian hơn. Với xu hướng phát triển công nghiệp, nhà cửa, xe cộ, máy móc có quy mô ngày càng lớn, số lượng công việc ngày càng nặng nhọc và phức tạp; sự tham gia của robot là một yếu tố tất yếu trong quá trình phát triển của xã hội. Và để phục vụ mục đích thay thế con người thực hiện những công việc nặng nhọc và nhàm chán, quá trình kết hợp một hệ thống đa robot cùng hoạt động thực hiện một công việc hay một chuỗi công việc, có khả năng phối hợp cùng nhau, hợp tác như con người và làm việc an toàn cùng con người là xu hướng phát triển ngành điều khiển robot trong thời đại mới.

1.1.2. Những thách thức trong lĩnh vực đa robot.

Tuy nhiên, lĩnh vực nghiên cứu đa robot đã và đang gặp rất nhiều thách thức khi giải quyết bài toán điều khiển nhiều robot đồng thời và cùng hợp tác làm một việc trong không gian làm việc chung nhỏ. Việc điều khiển một robot vận hành ổn định là một bài toán khó khăn cổ điển mà các nhà nghiên cứu vẫn đang tìm câu trả lời. Rất nhiều phương pháp được xây dựng dựa trên điều khiển tuần tự khi lập trình robot làm việc lần lượt từng công việc một, trong phương pháp này, người ta lên kế hoạch di chuyển cho robot theo một quỹ đạo từ điểm đầu đến từng mốc vị trí cần đi đến trong không gian để thực hiện công việc. Quá trình tính toán cho phương pháp trên rất phức tạp và tốn thời gian khi thực hiện bằng công cụ tính toán thô sơ, với từng

nhệm vụ khác nhau robot lại di chuyển một quỹ đạo khác nhau, dẫn đến không thể áp dụng một chương trình để giải quyết toàn bộ vấn đề mắc phải.

Sau một quá trình cải thiện phần cứng khi hệ thống máy tính phát triển nhanh chóng về khả năng tính toán, người ta đã xây dựng một số phương pháp tính toán nhanh với mạng nơ-ron tích xoắn chập, phương pháp học máy, học sâu, học tăng cường để đào tạo mô hình hoạt động lặp đi lặp lại một công việc nhiều lần khám phá qua cách thử thực hiện và đánh giá kết quả nào sai, kết quả nào tốt để phục vụ phương án đưa ra quyết định trong những lần sau của robot. Tuy nhiên, những phương pháp học trên một chính sách đưa quyết định dựa trên phân bố xác suất quyết định ngẫu nhiên khó có thể thực hiện tốt khi áp dụng vào mô hình hệ thống đa robot với quá nhiều trạng thái, tham số đầu vào. Hành động của robot là một chuỗi tuần tự cần thực hiện lần lượt từng xác suất lựa chọn trên mỗi bước trong chu trình di chuyển. Bài toán có môi trường cần sự thăm dò lớn, trạng thái đích đến chỉ có thể đạt được khi tuân theo một chuỗi các hành động nhất định trước đó, khó có thể giải quyết được khi hành động được lựa chọn ngẫu nhiên. Những phương pháp cơ bản bị giới hạn bởi nhiệm vụ thăm dò môi trường và lựa chọn hành động bị thiếu điểm thưởng, quá trình khám phá thường phải nhận điểm phạt khi không thực hiện được mục tiêu, không tìm được con đường đúng để đưa ra kết quả tốt.

1.2. Tổng quan về các bài toán trong hệ đa robot

Việc lập kế hoạch quỹ đạo cho robot thường có một trong hai hình thức: cách tiếp cận tách rời, trước tiên tìm ra một đường dẫn khả thi và sau đó tối ưu hóa việc điều khiển dọc theo đường dẫn đó; và phương pháp lập kế hoạch quỹ đạo trực tiếp, bao gồm việc áp dụng các ràng buộc động học của hệ thống để giải quyết một đường đi tối ưu. Hai phương pháp này đều có ưu điểm và nhược điểm. Một lượng nghiên cứu đáng kể đã tập trung vào các phương pháp cách cải thiện hiệu suất của hệ thống robot chỉ định di chuyển theo một quỹ đạo tính toán trước mà không xảy ra va chạm^{1 2}.

Những bài toán lớn trong hệ thống đa robot bao gồm việc giải quyết bài toán lập kế hoạch chuyển động rất phức tạp và công kênh cho hệ thống đa cánh tay robot với số lượng thông số đầu vào quá lớn không thể kiểm soát như số bậc tự do của từng cánh tay trong hệ, quỹ đạo của từng cánh tay, phân chia quá trình di chuyển làm sao cho hợp lý khi làm một nhiệm vụ nhất định cũng là nội dung đánh xem xét.

Với mỗi công việc nhất định, ta có từng bước thực hiện lần lượt cho từng công việc đó, hệ thống robot có thể phân chia các công việc một cách hợp lý để thực hiện đồng thời cùng nhau, làm giảm thời gian thực hiện của chu trình nhiệm vụ là một hướng phát triển tối ưu đem lại lợi ích rất lớn, đặc biệt cho các dây chuyền sản xuất công nghiệp làm việc liên tục với tần suất dày đặc. Khi tối ưu thời gian chạy dây chuyền cho mỗi nhiệm vụ một khoảng thời gian nhỏ khi hệ robot nhiều cánh tay cùng làm nhanh hơn để một mình cánh tay làm hết tất cả mọi thứ, thời gian cải thiện này giúp rút ngắn quá trình sản xuất, trong cùng một khoảng thời gian doanh nghiệp làm ra nhiều sản phẩm hơn, đem lại nguồn lợi ích trực tiếp.

Ngoài ra, bài toán di chuyển trong hệ như thế nào cho mỗi robot để chúng thực hiện được công việc của mình mà không cản trở quá trình làm việc của robot khác cũng là một vấn đề lớn. Bởi môi trường làm việc của hệ thống đa robot thường diễn ra trong một phạm vi nhỏ nhất định mà chắc chắn không gian làm việc của mỗi robot có khoảng giao nhau để cách robot cùng thao tác với nhau thực hiện nhiệm vụ. Nhưng trong phạm vi làm việc nhỏ, nguy cơ xảy ra va chạm là điều không thể tránh khỏi, và đây là một vấn đề nguy hiểm khi gây thiệt hại lớn mỗi hỏng hóc xảy ra lỗi va chạm giữa các robot. Vì vậy, đề tài chúng tôi thực hiện sẽ tập trung vào cải tiến cách hoạt động của hệ sao cho xác suất xảy ra va chạm trong hệ là nhỏ nhất.

Với các tiếp cận xác định quỹ đạo cho từng thành phần trong hệ, phương pháp này có thời gian tính toán rất lớn và khả năng ứng dụng khi thay đổi số lượng robot thêm/bớt vào hệ là không cao khi mỗi lần thay đổi quy mô của hệ sẽ phải tính toán lại tất cả mọi thứ từ đầu. Với xu hướng phát triển khi việc cộng tác giữa các robot và giữa robot và con người đang là xu hướng, phương pháp lập kế hoạch chuyển động trên đang không đáp ứng kịp thời khi nhu cầu giảm thiểu các bước tính toán thủ công của người lập trình cao, và khả năng tính toán của người lập trình không thể giải quyết được độ phức tạp cho hệ thống nhiều thông số đầu vào như bài toán đa robot. Thời gian thực hiện phép tính toán quỹ đạo chuyển động cho nhiều robot cũng diễn ra trong khoảng rất lâu, không đủ đáp ứng cho vấn đề nhiều robot thực hiện nhiều nhiệm vụ phức tạp. Vì vậy, đề tài đề xuất một phương án xây dựng mô hình học tăng cường có điểm thưởng cho toàn hệ để các robot cùng hợp tác đạt được điểm thưởng chung khi tất cả robot cùng hoàn thành nhiệm vụ của mình, không cản trở nhau.

1.3. Đặt vấn đề

Đề tài áp dụng phương pháp học tăng cường để huấn luyện mô hình hệ đa cánh tay robot thực hiện một số nhiệm vụ trên một bộ dữ liệu các tư thế, trạng thái của robot đã được tạo trước. Đề tài hướng tới mục tiêu nhằm giảm thời gian tính toán quỹ đạo chuyển động cho hệ thống gồm nhiều cánh tay robot cùng hoạt động đồng thời khi không sử dụng phương pháp điều khiển cổ điển phức tạp và không có tính áp dụng cao khi cần tính toán từng vị trí cho từng robot với từng quỹ đạo khác nhau. Khi sử dụng học tăng cường huấn luyện hành động của hệ thống robot trên tập dữ liệu các hành vi của robot có sẵn, các cánh tay trong hệ sẽ di chuyển một cách có quan sát tới các thành phần khác cùng làm việc trên môi trường để đi đến vị trí điểm thao tác cuối mong muốn của từng cánh tay mà không xảy ra va chạm lẫn nhau khi không gian làm việc của chúng có sự giao thoa.

Để cải thiện kết quả thực hiện, đề tài đề xuất áp dụng phương pháp đánh giá tối ưu Soft Actor-Critic vào mô hình học tăng cường để đưa ra cải thiện sau nhiều vòng lặp huấn luyện. Nhóm thực hiện thêm vào mô hình dữ liệu điều khiển mẫu dựa trên mô phạm hành động cho mỗi nhiệm vụ hệ robot không thực hiện được, từ đó so sánh kết quả giữa hai trường hợp và lựa chọn phương án có hiệu quả cao hơn để cập nhật chính sách lập kế hoạch chuyển động của mô hình robot.

1.4. Tổng quan nội dung đồ án.

Chương 1: khái quát về vấn đề và thách thức đang gặp phải với lĩnh vực nghiên cứu điều khiển hệ đa cánh tay robot hiện nay. Tại đây đề tài đưa ra phương hướng tiếp cận vấn đề thông qua xây dựng mô hình học tăng cường đa tác tử sử dụng phương pháp Soft Actor-Critic để đánh giá hoạt động của hệ, tại những trường hợp hệ robot thất bại trong thực hiện nhiệm vụ, ta thực hiện huấn luyện lại mô hình tại riêng những trường hợp đó với bộ dữ liệu kết quả cách thực hiện đúng cho nó.

Chương 2: Mô hình hóa toán học cho cánh tay robot UR5 sử dụng trong hệ thống đa cánh tay. Trong chương này chúng tôi xây dựng mô hình động học thuận, động học ngược, động lực học cho một cánh tay để áp dụng vào hệ thống điều khiển trong những chương sau.

Chương 3: Điều khiển cánh tay UR5 theo phương pháp điều khiển tập trung, sử dụng các mẫu điểm vị trí thao tác cuối để lập quỹ đạo chuyển động cho robot. Nội dung thực hiện trong chương này là phương pháp lấy mẫu dữ các mô phạm hành động thực hiện cho trường hợp hệ robot không hoàn thành nhiệm vụ để huấn luyện lại mô hình tính toán cho những trường hợp trên, cải thiện khả năng hoạt động của mô hình.

Chương 4: Phân tích các vấn đề trong lĩnh vực điều khiển hệ thống đa robot và trình bày các bước xây dựng bài toán của đề tài.

Chương 5: Thể hiện kết quả và đánh giá những công việc đã thực hiện được trong đề tài.

CHƯƠNG 2. MÔ HÌNH HÓA CÁN TAY ROBOT

Cho mô hình cánh tay robot như sau:



Hình 1.1. Mô hình cánh tay robot 6 bậc tự do.

a. Mô tả chi tiết các bước đặt các hệ tọa độ địa phương (theo gốc tọa độ cho trước) trên mô hình robot theo quy tắc DH và xác định bảng thông số DH. Hệ tọa độ cố định đã được cho trước.

b. Xây dựng các ma trận H và D . Xác định vị trí điểm thao tác cuối

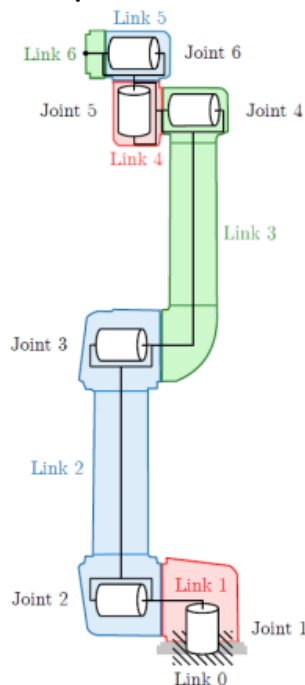
c. Tính ma trận Jacobian cho điểm thao tác. Tính định thức ma trận Jacobian.

d. Tìm vị trí điểm kì dị của cánh tay robot và vẽ hình minh họa các vị trí đó.

2.1. Mô hình hóa động học cho cánh tay robot.

2.1.1. Phân tích mô hình cánh tay 6 bậc tự do

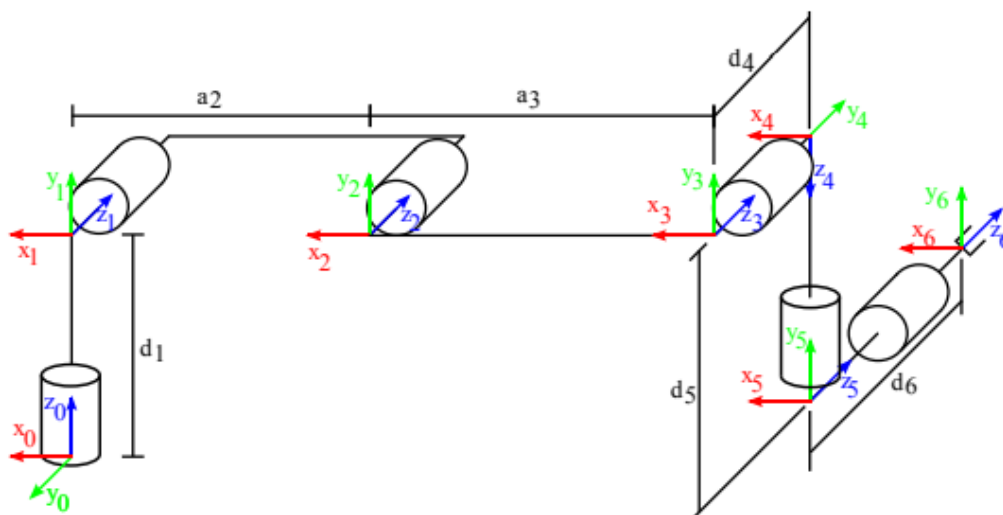
Mô hình cánh tay robot 6 bậc tự do được mô tả như hình 1.2



Hình 1.2. Mô hình cánh tay robot 2 bậc tự do.

Mô hình cánh tay UR5 gồm 6 khâu và 6 khớp tương ứng. Khớp 0 và khớp 4 là khớp xoay trục; khớp 1, khớp 2, khớp 3 và khớp 5 là khớp quay bản lề. Như vậy, cánh tay robot này chuyển động trong không gian ba chiều.

2.1.2. Đặt các hệ tọa độ cho mô hình.



Hình 1.3. Đặt các hệ tọa độ địa phương cho cánh tay robot

Đặt hệ tọa độ cố định $(Oxyz)_0$

Gốc O_0 đặt tại chân robot tại tâm xoay khớp 0 xoay quanh trục $(Oz)_0$.

Các trục $(Oz)_0$ và $(Ox)_0$ được đặt như hình vẽ.

Đặt hệ tọa độ $(Oxyz)_1$

Khớp 1 là khớp xoay bản lề với biến khớp $q_1(rad)$ nên trục quay của khớp 1 đi qua tâm của khớp và vuông góc với hai khâu: khâu 1 và 2.

Ta đặt gốc tọa độ O_1 tại tâm quay của khớp 1 và cách gốc O_0 một đoạn $d_1(m)$

Trục $(Oz)_1$ nằm trên trục quay của khớp 1, có chiều hướng ra ngoài như hình vẽ.

Trục $(Ox)_1$ đặt song song với trục $(Ox)_0$ như hình vẽ.

Đặt hệ tọa độ $(Oxyz)_2$

Khớp 2 là khớp quay bản lề với biến khớp $q_2(rad)$, trục quay của khớp 2 đi qua tâm của khớp và hướng ra ngoài, vuông góc với hai khâu: khâu 1 và khâu 2.

Đặt gốc tọa độ O_2 tại tâm quay của khớp 2.

Trục $(Oz)_2$ nằm trên trục quay của khớp 2, có chiều hướng ra ngoài như hình vẽ.

Trục $(Ox)_2$ nằm dọc theo chiều dài khâu 2 hay đường vuông góc chung của 2 trục $(Oz)_1$ và $(Oz)_2$. Ta chọn trục $(Ox)_2$ nằm ở đường kéo dài của khâu 2, có chiều hướng lên trên như hình.

Đặt hệ tọa độ $(Oxyz)_3$

Khớp 3 là khớp quay bản lề với biến khớp $q_3(rad)$, trục quay của khớp 3 đi qua tâm của khớp và hướng ra ngoài, vuông góc với hai khâu: khâu 2 và khâu 3.

Đặt gốc tọa độ O_3 tại tâm quay của khớp 3.

Trục $(Oz)_3$ nằm trên trục quay của khớp 3, có chiều hướng ra ngoài như hình vẽ.

Trục $(Ox)_3$ nằm dọc theo chiều dài khâu 3 hay đường vuông góc chung của 2 trục $(Oz)_2$ và $(Oz)_3$. Ta chọn trục $(Ox)_3$ nằm ở đường kéo dài của khâu 3, có chiều hướng lên trên như hình.

Đặt hệ tọa độ $(Oxyz)_4$

Khớp 4 là khớp quay trục với biến khớp $q_4(rad)$, trục quay của khớp 4 đi qua tâm của khớp 4 và hướng ra ngoài, vuông góc với hai khâu: khâu 3 và khâu 5.

Đặt gốc tọa độ O_4 tại tâm quay của khớp 4.

Trục $(Oz)_4$ nằm trên trục quay của khớp 4, có chiều hướng ra ngoài như hình vẽ.

Trục $(Ox)_4$ song song với trục $(Ox)_3$ và cách trục $(Ox)_3$ một đoạn d_4

Đặt hệ tọa độ $(Oxyz)_5$

Khớp 5 là khớp quay bản lề với biến khớp $q_5(rad)$, trục quay của khớp 5 đi qua tâm của khớp 5 và hướng ra ngoài, vuông góc với hai khâu: khâu 4 và khâu 6.

Đặt gốc tọa độ O_5 tại tâm quay của khớp 5.

Trục $(Oz)_5$ nằm trên trục quay của khớp 5, có chiều hướng ra ngoài như hình vẽ.

Trục $(Ox)_5$ song song với trục $(Ox)_4$ và cách trục $(Ox)_4$ một đoạn d_5

Đặt hệ tọa độ $(Oxyz)_6$ tại điểm thao tác E

Ta đặt gốc tọa độ O_6 tại điểm E.

Đặt trục $(Oz)_6$ có phương song song với phương của trục $(Oz)_5$, cùng hướng với trục $(Oz)_5$

Đặt trục $(Ox)_6$ song song với trục $(Ox)_5$ và cách trục $(Ox)_5$ một đoạn d_6 , hướng như hình.

Các trục $(Oy)_i$ ta có thể dễ dàng xác định bằng quy tắc bàn tay phải để các hệ tọa độ $(Oxyz)_i$ là hệ quy chiếu thuận.

2.2.3. Xây dựng bảng DH

Quy tắc DH được tuân thủ cho khớp i để dịch chuyển hệ tọa độ $(Oxyz)_{i-1}$ sang hệ tọa độ $(Oxyz)_i$ bất kỳ theo nguyên tắc:

Quay trục $(Oz)_{i-1}$ với góc θ_{i-1} .

Tịnh tiến trục $(Oz)_{i-1}$ với chiều dài d_{i-1} .

Tịnh tiến trục $(Ox)_{i-1}$ với chiều dài a_{i-1}

Quay trục $(Ox)_{i-1}$ với góc α_{i-1} .

Chuyển đổi từ hệ $(Oxyz)_0$ sang hệ $(Oxyz)_1$

Xoay trục $(Oz)_0$ nên $\theta = q_1$.

Tịnh tiến trục $(Oz)_0$ để $(Ox)_0 \rightarrow (Ox)_1$ nên $d = d_1$.

$(Oz)_0$ và $(Ox)_1$ giao nhau tại O_1 nên không cần tịnh tiến trục $(Ox)_1$, $a = 0$.

Xoay $(Ox)_1$ một góc $\frac{\pi}{2}$ để đưa trục $(Oz)_0$ về trục $(Oz)_1$ nên $\alpha = \frac{\pi}{2}$.

Vậy: $\theta = q_1$, $d = d_1$, $a = 0$, $\alpha = \frac{\pi}{2}$.

Chuyển đổi từ hệ $(Oxyz)_1$ sang hệ $(Oxyz)_2$

Xoay trục $(Oz)_1$ một góc $\theta = q_2$ để đưa phương của trục $(Ox)_1$ trùng phương với $(Ox)_2$.

$(Oz)_1$ và $(Ox)_2$ giao nhau tại O_1 nên không cần tịnh tiến trục $(Oz)_1$, $d = 0$.

$(Oz)_1$ và $(Ox)_2$ giao nhau tại O_2 , cần tịnh tiến trục $(Ox)_2$ để $O_1 \equiv O_2$, $a = -a_2$.

Trục $(Oz)_1$ về trục $(Oz)_2$ có phương song song nên không cần xoay trục $(Ox)_2 \rightarrow \alpha_2 = 0$

Vậy: $\theta = q_2$, $d = 0$, $a = -a_2$, $\alpha = 0$.

Chuyển đổi từ hệ $(Oxyz)_2$ sang hệ $(Oxyz)_3$

Xoay trục $(Oz)_2$ một góc $\theta = q_3$ để đưa phương của trục $(Ox)_2$ trùng phương với $(Ox)_3$.

$(Oz)_2$ và $(Ox)_3$ giao nhau tại O_2 nên không cần tịnh tiến trục $(Oz)_2$, $d = 0$.

$(Oz)_2$ và $(Ox)_3$ giao nhau tại O_3 , cần tịnh tiến trục $(Ox)_3$ để $O_2 \equiv O_3(E)$ vậy $a = -a_3$.

Trục $(Oz)_3$ về trục $(Oz)_2$ có phương song song nên không cần xoay trục $(Ox)_3 \rightarrow \alpha_3 = 0$.

Vậy: $\theta = q_3$, $d = 0$, $a = -a_3$, $\alpha = 0$.

Chuyển đổi từ hệ $(Oxyz)_3$ sang hệ $(Oxyz)_4$

Xoay trục $(Oz)_3$ một góc $\theta = q_4$ để đưa phương của trục $(Ox)_3$ trùng phương với $(Ox)_4$.

$(Ox)_3$ và $(Oz)_4$ giao nhau tại O_4 nên cần tịnh tiến trục $(Oz)_3$, $d = d_4$.

$(Oz)_3$ và $(Ox)_4$ giao nhau tại O_4 , không cần tịnh tiến trục $(Ox)_3$ vậy $a = 0$.

Trục $(Oz)_3$ vuông góc trục $(Oz)_4$ cần xoay trục $(Ox)_4 \rightarrow \alpha = \frac{\pi}{2}$.

Vậy: $\theta = q_4$, $d = d_4$, $a = 0$, $\alpha = \frac{\pi}{2}$.

Chuyển đổi từ hệ $(Oxyz)_4$ sang hệ $(Oxyz)_5$

Xoay trục $(Oz)_4$ một góc $\theta = q_5$ để đưa phương của trục $(Ox)_4$ trùng phương với $(Ox)_5$.

$(Ox)_4$ và $(Oz)_5$ giao nhau tại O_4 nên cần tịnh tiến trục $(Oz)_4$, $d = d_5$.

$(Oz)_4$ và $(Ox)_5$ giao nhau tại O_5 , không cần tịnh tiến trục $(Ox)_4$ vậy $a = 0$.

Trục $(Oz)_4$ vuông góc trục $(Oz)_5$ cần xoay trục $(Ox)_4 \rightarrow \alpha = -\frac{\pi}{2}$.

Vậy: $\theta = q_5$, $d = d_5$, $a = 0$, $\alpha = -\frac{\pi}{2}$.

Chuyển đổi từ hệ $(Oxyz)_4$ sang hệ $(Oxyz)_5$

Xoay trục $(Oz)_3$ một góc $\theta = q_6$ để trục $(Ox)_5$ trùng với phương của trục $(Ox)_6$.

$(Ox)_5$ và $(Oz)_6$ giao nhau tại O_6 nên cần tịnh tiến trục $(Oz)_5$, $d = d_6$.

$(Oz)_5$ và $(Ox)_6$ giao nhau tại O_6 , không cần tịnh tiến trục $(Ox)_5$ vậy $a = 0$.

Trục $(Oz)_5$ trùng phương trục $(Oz)_6 \rightarrow \alpha = 0$.

Vậy: $\theta = q_6$, $d = d_5$, $a = 0$, $\alpha = 0$

Các thông số DH được mô tả trong bảng 1.

Bảng 1. Các thông số DH

Dịch chuyển	Khâu i	θ_i	d_i	a_i	α_i
$O_0 \Rightarrow O_1$	1	q_1	d_1	0	$\frac{\pi}{2}$
$O_1 \Rightarrow O_2$	2	q_2	0	$-a_2$	0
$O_2 \Rightarrow O_3$	3	q_3	0	$-a_3$	0
$O_2 \Rightarrow O_3$	4	q_4	d_4	0	$\frac{\pi}{2}$
$O_2 \Rightarrow O_3$	5	q_5	d_5	0	$-\frac{\pi}{2}$
$O_5 \Rightarrow O_6(E)$	6	q_6	d_6	0	0

2.2.4. Tính các ma trận H .

Việc tính các ma trận H_i và ma trận D_i được thực hiện trên các phần mềm Maple và Matlab. Quy ước rằng:

$$\cos q_1 = c_1, \cos q_2 = c_2, \cos q_3 = c_3, \cos(q_1 + q_2) = c_{12}, \cos(q_1 - q_2) = c_{1-2},$$

$$\sin q_1 = s_1, \sin q_2 = s_2, \sin q_3 = s_3, \sin(q_1 + q_2) = s_{12}, \sin(q_1 - q_2) = s_{1-2},$$

Áp dụng tương tự cho các khâu 2, khâu 3.

Công thức tổng quát của ma trận H_i

$${}^i{}_{i-1}\mathbf{H} = \mathbf{H}_i = \begin{bmatrix} c\theta_i & -s\theta_i c\alpha_i & s\theta_i s\alpha_i & a_i c\theta_i \\ s\theta_i & c\theta_i c\alpha_i & -c\theta_i s\alpha_i & a_i s\theta_i \\ 0 & s\alpha_i & c\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.1)$$

Ma trận $\mathbf{H}_{01}$ chuyển đổi hệ tọa độ $(Oxyz)_0$ sang hệ $(Oxyz)_1$:

$$\mathbf{H}_{01} = \begin{bmatrix} c_1 & 0 & s_1 & 0 \\ s_1 & 0 & -c_1 & 0 \\ 0 & 1 & 0 & d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.2)$$

Ma trận $\mathbf{H}_{12}$ chuyển đổi hệ tọa độ $(Oxyz)_1$ sang hệ $(Oxyz)_2$:

$$\mathbf{H}_{12} = \begin{bmatrix} c_2 & -s_2 & 0 & -a_2 c_2 \\ s_2 & c_2 & 0 & -a_2 s_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.3)$$

Ma trận $\mathbf{H}_{23}$ chuyển đổi hệ tọa độ $(Oxyz)_2$ sang hệ $(Oxyz)_3$:

$$\mathbf{H}_{23} = \begin{bmatrix} c_3 & -s_3 & 0 & -a_3 c_3 \\ s_3 & c_3 & 0 & -a_3 s_3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.4)$$

Ma trận $\mathbf{H}_{34}$ chuyển đổi hệ tọa độ $(Oxyz)_3$ sang hệ $(Oxyz)_4$:

$$\mathbf{H}_{34} = \begin{bmatrix} c_4 & 0 & s_4 & 0 \\ s_4 & 0 & -c_4 & 0 \\ 0 & 0 & 1 & d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.5)$$

Ma trận $\mathbf{H}_{45}$ chuyển đổi hệ tọa độ $(Oxyz)_4$ sang hệ $(Oxyz)_5$:

$$\mathbf{H}_{45} = \begin{bmatrix} c_5 & 0 & -s_5 & 0 \\ s_5 & 0 & c_5 & 0 \\ 0 & -1 & 0 & d_5 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.6)$$

Ma trận $\mathbf{H}_{56}$ chuyển đổi hệ tọa độ $(Oxyz)_5$ sang hệ $(Oxyz)_6$:

$$\mathbf{H}_{56} = \begin{bmatrix} c_6 & -s_6 & 0 & 0 \\ s_6 & c_6 & 0 & 0 \\ 0 & 0 & 1 & d_6 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.7)$$

2.2.5. Tính các ma trận D.

Ma trận chuyển đổi thuần nhất từ hệ $(Oxyz)_i$ so với hệ tọa độ cố định $(Oxyz)_0$ được xác định như sau:

Ma trận D_{01} mô tả hướng hệ $(Oxyz)_1$ và tọa độ gốc O_1 so với hệ $(Oxyz)_0$

$$D_{01} = H_{01} = \begin{bmatrix} c_1 & 0 & s_1 & 0 \\ s_1 & 0 & -c_1 & 0 \\ 0 & 1 & 0 & d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.8)$$

Ma trận D_{02} mô tả hướng khâu 2, hệ $(Oxyz)_2$ và tọa độ gốc O_2 so với hệ $(Oxyz)_0$

$$D_{02} = H_{01} H_{12} = \begin{bmatrix} c_1 c_2 & -c_1 s_2 & s_1 & -a_2 c_1 c_2 \\ s_1 c_2 & -s_1 s_2 & -c_1 & -a_2 s_1 c_2 \\ s_2 & c_2 & 0 & -a_2 s_2 + d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.9)$$

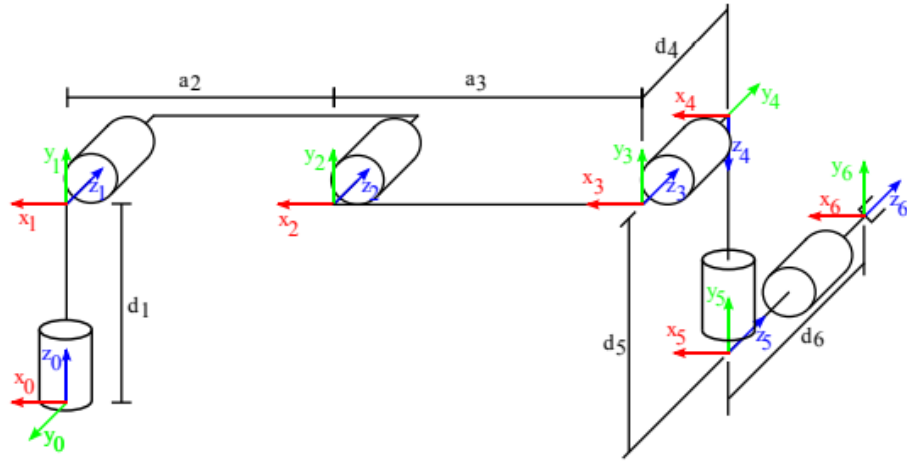
Ma trận D_{03} mô tả hướng khâu 3, hệ $(Oxyz)_3$ và tọa độ gốc O_3 so với hệ $(Oxyz)_0$. Ma trận D_{04} mô tả hướng khâu 4, hệ $(Oxyz)_4$ và tọa độ gốc O_4 so với hệ $(Oxyz)_0$. Ma trận D_{05} mô tả hướng khâu 5, hệ $(Oxyz)_5$ và tọa độ gốc O_5 so với hệ $(Oxyz)_0$. Ma trận D_{06} mô tả hướng khâu 6, hệ $(Oxyz)_6$ và tọa độ gốc O_6 so với hệ $(Oxyz)_0$. Vì điểm E trùng với gốc tọa độ O_6 , nên ma trận D_{0E} mô tả hướng khâu thao tác (bàn kẹp) và tọa độ điểm E so với hệ $(Oxyz)_0$ là ma trận D_{06} . Tuy nhiên vì phương trình tính toán phức tạp nên phần trình bày mô hình các ma trận chuyển hướng của các khâu 3, 4, 5, 6 và phương trình động lực học của cánh tay robot 6 bậc tự do sẽ được trình bày trong file Maple “DH_DLH_RB6DOF_NHT.mw”

2.2.6. Kiểm tra hệ tọa độ điểm thao tác.

Ta xét vector $\mathbf{q} = [q_1 \quad q_2 \quad q_3 \quad q_4 \quad q_5 \quad q_6]$ là vector góc khớp có chứa tất cả các thông số góc khớp biểu diễn vị trí, dáng của robot tại một trạng thái. Kiểm tra tọa độ điểm cuối của cánh tay cho một số trường hợp sau:

Vị trí 1: $\mathbf{q}_1 = [0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0]$

Hình dáng robot xác định bằng hình học như hình 1.4.



Hình 1.4. Mô hình Robot ở vị trí 1

Tọa độ điểm E trong hệ tọa độ $(Oxyz)_0$ xác định bằng hình học:

$$E = \begin{bmatrix} x_E \\ y_E \\ z_E \end{bmatrix} = \begin{bmatrix} -a_3 - a_2 \\ -d_6 - d_4 \\ -d_5 + d_1 \end{bmatrix} \quad (2.10)$$

Tọa độ điểm E xác định bằng tính toán trong Maple:

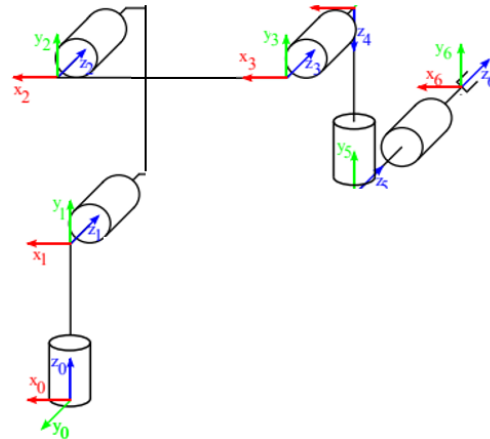
$$\left[\begin{array}{l} > E0 = \text{simplify}(\text{subs}(\{q1 = 0, q2 = 0, q3 = 0, q4 = 0, q5 = 0, q6 = 0\}, E)); \\ & E0 = \begin{bmatrix} -a3 - a2 \\ -d6 - d4 \\ -d5 + d1 \end{bmatrix} \end{array} \right]$$

Hình 1.5. Biểu diễn tọa độ điểm E tính bằng Maple.

Như vậy, tọa độ điểm E được xác định là trùng khớp

Vị trí 2: $q_2 = \begin{bmatrix} 0 & -\frac{\pi}{2} & \frac{\pi}{2} & 0 & 0 & 0 \end{bmatrix}$

Hình dáng robot xác định bằng hình học như hình 1.5



Hình 1.6. Mô hình Robot ở vị trí 2

Tọa độ điểm E trong hệ tọa độ $(Oxyz)_0$ xác định bằng hình học:

$$E_1 = \begin{bmatrix} -a_3 \\ -d_6 - d_4 \\ -d_5 + a_2 + d_1 \end{bmatrix} \quad (2.11)$$

Tọa độ điểm E xác định bằng tính toán trong Maple:

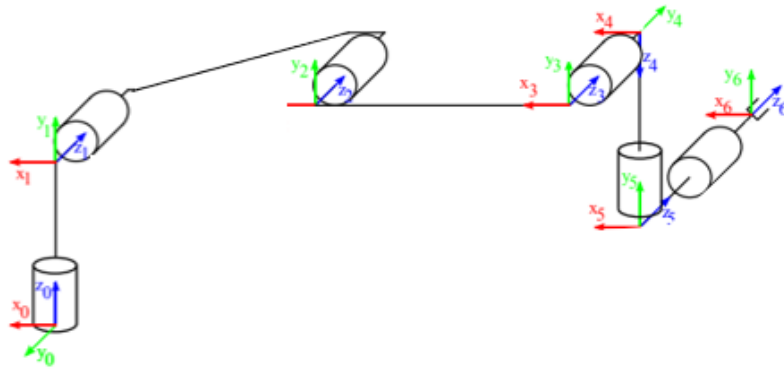
$$\left[\begin{array}{l} > E1 = \text{simplify} \left(\text{subs} \left(\left\{ q1 = 0, q2 = -\frac{\pi}{2}, q3 = \frac{\pi}{2}, q4 = 0, q5 = 0, q6 = 0 \right\}, E \right) \right); \\ \\ E1 = \begin{bmatrix} -a3 \\ -d6 - d4 \\ -d5 + a2 + d1 \end{bmatrix} \end{array} \right];$$

Hình 1.7. Kết quả tính tọa độ điểm thao tác cuối ở vị trí 2 với Maple.

Như vậy, tọa độ điểm E được xác định là trùng khớp.

Vị trí 3: $q_2 = \begin{bmatrix} 0 & -\frac{\pi}{6} & \frac{\pi}{6} & 0 & 0 & 0 \end{bmatrix}$

Hình dáng robot xác định bằng hình học như hình 1.6.



Hình 1.8. Mô hình Robot ở vị trí 3

Tọa độ điểm E trong hệ tọa độ $(Oxyz)_0$ xác định bằng hình học:

$$E_2 = \begin{bmatrix} -a_3 - \frac{\sqrt{3}}{2} a_2 \\ -d_6 - d_4 \\ -d_5 + \frac{a_2}{2} + d_1 \end{bmatrix} \quad (2.12)$$

Tọa độ điểm E xác định bằng tính toán trong Maple:

$$E2 = \begin{bmatrix} -a3 - \frac{a2\sqrt{3}}{2} \\ -d6 - d4 \\ -d5 + \frac{a2}{2} + d1 \end{bmatrix}$$

Hình 1.9. Biểu diễn tọa độ điểm E tính bằng Maple.

Như vậy, tọa độ điểm E được xác định là trùng khớp. Phương trình biểu diễn mô hình động học thuận của cánh tay robot là đúng đắn.

2.2. Mô hình hóa động lực học cho cánh tay robot.

a. Xác định các thành phần của phương trình động lực học trong không gian khớp: ma trận khối lượng $\mathbf{M}(\mathbf{q})$, ma trận Coriolis $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$, ma trận thế năng trọng trường $\mathbf{G}(\mathbf{q})$, véc tơ lực/mô men suy rộng τ .

b. Xác định các thành phần của phương trình động lực học trong không gian thao tác: ma trận khối lượng $\mathbf{M}_x$, ma trận Coriolis $\mathbf{C}_x$, ma trận thế năng trọng trường $\mathbf{G}_x$, véc tơ lực/mô men suy rộng $\mathbf{F}_x$.

2.2.1. Ma trận cosin chỉ hướng của các khâu theo hệ tọa độ cố định

Ma trận cosin chỉ hướng khâu 0 – hệ tọa độ 1 với 0

$$\mathbf{A}_1 = \begin{bmatrix} c_1 & 0 & s_1 \\ s_1 & 0 & -c_1 \\ 0 & 1 & 0 \end{bmatrix} \quad (2.13)$$

Ma trận cosin chỉ hướng khâu 1

$$\mathbf{A}_2 = \begin{bmatrix} c_1 c_2 & -c_1 s_2 & s_1 \\ s_1 c_2 & -s_1 s_2 & -c_1 \\ s_1 & c_2 & 0 \end{bmatrix} \quad (2.14)$$

Vì biểu diễn phép tính quá phức tạp, các ma trận cosin chỉ hướng khâu 2, khâu 3, khâu 4, khâu 5 nhóm xin phép được trình bày trong file tính toán bằng Maple dưới đây: “DH_DLH_RB6DOF_NHT.mw” [[link](#)].

2.2.2. Tọa độ điểm cuối các khâu theo hệ tọa độ cố định

Tọa độ điểm đầu khâu 1

$$\mathbf{x}_0 = [0 \quad 0 \quad d_1]^T \quad (2.15)$$

Tọa độ điểm cuối khâu 2

$$\mathbf{x}_1 = \begin{bmatrix} -a_2 c_1 c_2 \\ -a_2 s_1 s_2 \\ -a_2 s_1 + d_1 \end{bmatrix} \quad (2.16)$$

Tọa độ điểm cuối khâu 2, khâu 3, khâu 4, khâu 5 nhóm xin phép được trình bày trong file tính toán bằng Maple dưới đây: “DH_DLH_RB6DOF_NHT.mw” [[link](#)].

2.2.3. Tính tọa độ trọng tâm các khâu theo hệ tọa độ địa phương

Tọa độ trọng tâm khâu 1

$$\mathbf{x}_{G1}^{(0)} = \begin{bmatrix} 0 & 0 & \frac{d_1}{2} \end{bmatrix}^T \quad (2.17)$$

Tọa độ trọng tâm khâu 2

$$\mathbf{x}_{G2}^{(1)} = \begin{bmatrix} \frac{-a_2 c_2}{2} & \frac{-a_2 s_2}{2} & 0 \end{bmatrix}^T \quad (2.18)$$

Tọa độ trọng tâm khâu 3

$$\mathbf{x}_{G3}^{(2)} = \begin{bmatrix} \frac{-a_3 c_3}{2} & \frac{-a_3 s_3}{2} & 0 \end{bmatrix}^T \quad (2.19)$$

Tọa độ trọng tâm khâu 4

$$\mathbf{x}_{G4}^{(3)} = \begin{bmatrix} 0 & 0 & \frac{d_4}{2} \end{bmatrix}^T \quad (2.20)$$

Tọa độ trọng tâm khâu 5

$$\mathbf{x}_{G5}^{(4)} = \begin{bmatrix} 0 & 0 & \frac{d_5}{2} \end{bmatrix}^T \quad (2.21)$$

Tọa độ trọng tâm khâu 6

$$\mathbf{x}_{G6}^{(5)} = \begin{bmatrix} 0 & 0 & \frac{d_6}{2} \end{bmatrix}^T \quad (2.22)$$

2.2.4. Tính tọa độ trọng tâm các khâu theo hệ tọa độ cố định

Tọa độ trọng tâm khâu 1

$$\mathbf{x}_{G1}^{(0)} = \begin{bmatrix} 0 & 0 & \frac{d_1}{2} \end{bmatrix}^T \quad (2.23)$$

Tọa độ trọng tâm khâu 2

$$\mathbf{x}_{G2}^{(0)} = \begin{bmatrix} -\frac{a_2 c_1 c_2}{2} \\ -\frac{a_2 s_1 s_2}{2} \\ -\frac{a_2 s_2}{2} + d_1 \end{bmatrix} \quad (2.24)$$

Tọa độ trọng tâm khâu 3, khâu 3, khâu 4, khâu 5 nhóm xin phép được trình bày trong file tính toán bằng Maple dưới đây: “DH_DLH_RB6DOF_NHT.mw” [\[link\]](#).

2.2.5. Ma trận khối lượng M.

Ma trận khối lượng có dạng:

$$\mathbf{M} = \begin{bmatrix} M_{11} & M_{12} & \cdot & \cdot & \cdot & M_{16} \\ M_{21} & M_{22} & & & & \\ \cdot & & \cdot & & & \\ \cdot & & & \cdot & & \\ \cdot & & & & \cdot & \\ M_{61} & & & & & M_{66} \end{bmatrix} \quad (2.25)$$

Trong đó, các phần tử của ma trận M xin phép được trình bày trong file tính toán bằng Maple dưới đây: “DH_DLH_RB6DOF_NHT.mw” [\[link\]](#).

2.2.6. Ma trận Coriolis C.

Ma trận Coriolis có dạng

$$\mathbf{C} = \begin{bmatrix} CC_{11} & CC_{12} & \cdot & \cdot & \cdot & CC_{16} \\ CC_{21} & CC_{22} & & & & \\ \cdot & & \cdot & & & \\ \cdot & & & \cdot & & \\ \cdot & & & & \cdot & \\ CC_{61} & & & & & CC_{66} \end{bmatrix} \quad (2.26)$$

Trong đó, các phần tử của ma trận C được tính trong phần “Ma trận Coriolis C” tại mục ‘CC’ nhóm xin phép được trình bày trong file tính toán bằng Maple dưới đây: “DH_DLH_RB6DOF_NHT.mw” [\[link\]](#).

2.2.7. Ma trận thế năng trọng trường G

Ma trận thế năng trọng trường có dạng:

$$\mathbf{G} = [G_{11} \quad G_{21} \quad G_{31} \quad G_{41} \quad G_{51} \quad G_{61}]^T \quad (2.27)$$

Trong đó, các phần tử của ma trận G được tính trong phần “Ma trận Coriolis C” nhóm xin phép được trình bày trong file tính toán bằng Maple dưới đây: “DH_DLH_RB6DOF_NHT.mw” [\[link\]](#).

2.3. Mô hình hóa động lực học trong không gian thao tác.

Véc tơ mô tả vị trí điểm:

$$\mathbf{x} = [x \quad y \quad z]^T \quad (2.28)$$

Véc tơ mô tả vị trí các khớp:

$$\mathbf{q} = [q_1 \quad q_2 \quad \dots \quad q_n]^T \quad (2.29)$$

Véc tơ lực/mômen suy rộng:

$$\boldsymbol{\tau} = [F_1 \quad \tau_2 \quad \tau_3]^T \quad (2.30)$$

Phương trình động lực học trong không gian khớp được mô tả như sau:

$$\mathbf{M}\ddot{\mathbf{q}} + \mathbf{C}\dot{\mathbf{q}} + \mathbf{G} = \boldsymbol{\tau} \quad (2.31)$$

Phương trình động học tổng quát:

$$\mathbf{x} = \mathbf{f}(\mathbf{q}) \quad (2.32)$$

Phương trình vận tốc điểm thao tác:

$$\dot{\mathbf{x}} = \mathbf{J}\dot{\mathbf{q}} \quad (2.33)$$

Trong đó, $\mathbf{J}$ là ma trận Jacobian.

Phương trình gia tốc điểm thao tác:

$$\ddot{\mathbf{x}} = \dot{\mathbf{J}}\dot{\mathbf{q}} + \mathbf{J}\ddot{\mathbf{q}} \quad (2.34)$$

Phương trình vận tốc biên khớp được xác định:

$$\dot{\mathbf{q}} = \mathbf{J}^{-1}\dot{\mathbf{x}} \quad (2.35)$$

Phương trình gia tốc biên khớp được xác định:

$$\ddot{\mathbf{q}} = \mathbf{J}^{-1}(\ddot{\mathbf{x}} - \dot{\mathbf{J}}\dot{\mathbf{q}}) \quad (2.36)$$

Phương trình động lực học trong không gian thao tác:

$$\mathbf{M}_x \ddot{\mathbf{x}} + \mathbf{C}_x \dot{\mathbf{x}} + \mathbf{G}_x = \mathbf{F}_x \quad (2.37)$$

Trong đó:

$$\mathbf{M}_x = \mathbf{M}\mathbf{J}^{-1} \quad (2.38)$$

$$\mathbf{C}_x = \mathbf{C}\mathbf{J}^{-1} - \mathbf{M}\mathbf{J}^{-1}\dot{\mathbf{J}}\mathbf{J}^{-1} \quad (2.39)$$

$$\mathbf{G}_x = \mathbf{G} \quad (2.40)$$

$$\mathbf{F}_x = \mathbf{J}^T \boldsymbol{\tau} \quad (2.41)$$

Với $\mathbf{J}^T$ là ma trận chuyển vị của $\mathbf{J}$.

Phần tính toán chi tiết phương trình động lực học trong không gian thao tác được trình bày ở file Matlab và Maple.

CHƯƠNG 3. ĐIỀU KHIỂN TẬP TRUNG

3.1. Cơ sở lý thuyết

Thông thường, quá trình điều khiển tập trung dựa trên lập trình quỹ đạo di chuyển cho robot sử dụng các vị trí điểm thao tác cuối của cánh tay để tính toán động học ngược, từ đó xác định giá trị các góc khớp phù hợp phục vụ chu trình di chuyển. Ta cần xây dựng mô hình động học thuận dựa trên phương pháp Denavit - Hartenberg và xây dựng mô hình động lực học bằng phương pháp Euclidean – Lagrangian.

Để quỹ đạo di chuyển robot diễn ra một cách hợp lý và ổn định, ta cần xác định rất nhiều tham số ràng buộc cho vị trí mục tiêu khi mục tiêu cần phải nằm trên một chuỗi hành động tại đó robot thực hiện lần lượt và tuần tự, tại đây ta có các ràng buộc về vận tốc và gia tốc tại những điểm mốc trong chu trình di chuyển. Thực hiện lập phương trình biến đổi về quãng đường chuyển dịch giữa các mốc, xác định vận tốc và gia tốc cho quãng đường đó với phép đạo hàm phương trình động học thuận của điểm thao tác cuối.

Dựa trên những ràng buộc ta đã xác định khi đánh dấu các điểm trên quỹ đạo, ta giải hệ phương trình để thu được các hằng số cho phương trình quỹ đạo chuyển dịch tại điểm thao tác cuối. Từ đây ta tìm được giá trị góc khớp và vận tốc góc khớp để truyền tín hiệu điều khiển cho mỗi động cơ tại mỗi khớp trong cánh tay di chuyển đến vị trí mong muốn.

3.2. Xác định bài toán.

Ví dụ này minh họa việc lập kế hoạch và thực hiện chuyển động cho ứng dụng phân phối ràng buộc bằng cobot. Thuật toán quỹ đạo dựa trên không gian nhiệm vụ được minh họa trong ví dụ này bằng cách sử dụng các tính năng của Robotics System Toolbox™. Việc lập kế hoạch quỹ đạo đảm bảo rằng chúng ta có được vận tốc tuyến tính TCP - Tool Center Point (Điểm trung tâm công cụ) xấp xỉ không đổi để phân phối ràng buộc đồng đều trên đối tượng kính chắn gió (được coi là đối tượng tham chiếu cho phân trình diễn).

Trong ví dụ này, trước tiên, chúng tôi sẽ sử dụng các mô hình cây cứng - rigid body tree models để lập kế hoạch và mô phỏng quỹ đạo trong MATLAB, sau đó chúng tôi sẽ sử dụng trình mô phỏng vật lý Gazebo và đối tượng Universalrobot để xác thực khả năng điều khiển và chuyển động của cobot trên quỹ đạo không gian nhiệm vụ được tính toán trước. Tất cả các tệp cần thiết để thiết lập Gazebo đều được bao gồm trong gói hỗ trợ. Tham khảo Cài đặt gói và phụ thuộc ROS cho ROS để biết thêm chi tiết.

Xóa tất cả đầu ra trong tập lệnh trực tiếp bằng cách nhấp chuột phải vào bất kỳ vị trí nào trong tập lệnh trực tiếp rồi nhấp vào 'Xóa tất cả đầu ra' hoặc điều hướng đến XEM > Xóa tất cả đầu ra. Các thư viện | packages cần thiết để chạy chương trình.

- Robotics System Toolbox™
- Robotics System Toolbox™ Support Package for Manipulators
- ROS Toolbox

3.3. Mô phỏng hoạt động cánh tay robot.

3.3.1. Thiết lập mô hình robot.

Ví dụ này sử dụng mô hình cobot Universal Robot UR5, một bộ điều khiển robot 6 bậc tự do. Loadrobot tạo ra mô hình RigidBodyTree từ mô tả được lưu trữ trong tệp Định dạng Mô tả Robot Hợp nhất (URDF). Tải RigidBodyTreeModels của Universal Robot UR5.

```
ur5_RBT = loadrobot("universalUR5");  
ur5_RBT.DataFormat = 'row';
```

Thêm dummy tại điểm thao tác cuối trên thân robot UR5: thêm tay gấp tại điểm tác động cuối của cobot bằng addBody. Hình ảnh có thể được addVisuals thêm vào bằng tệp STL. Các chức năng tương tác RigidBodyTree được sử dụng trong ví dụ này để đặt hướng của bộ tác động cuối bằng cách sử dụng nội dung đánh dấu. Xác định hàm nội tuyến để xoay quanh trục x để sử dụng trong các phần mã tiếp theo.

```
rotationX = @(t) [1 0 0; 0 cosd(t) -sind(t) ; 0 sind(t) cosd(t)] ;
```

Chương trình thêm tay gấp vào mô hình robot như sau:

```
glueDispenserBody = rigidBody('dispenser');  
addVisual(glueDispenserBody,"Mesh",'glueDispenserMesh.STL')  
glueDispenserBodyJoint = rigidBodyJoint('glueDispenserBodyJoint','fixed');  
glueDispenserBody.Joint = glueDispenserBodyJoint;  
transfForglueDispenserBody = rotm2tform(rotationX(-90));  
setFixedTransform(glueDispenserBody.Joint, transfForglueDispenserBody)  
curEndEffectorBodyName = ur5_RBT.BodyNames{10};  
addBody(ur5_RBT,glueDispenserBody,curEndEffectorBodyName)
```

Xác định điểm cố định trên tay gấp phục vụ cho chương trình tính động học ngược ở phần sau bằng cách:

```
transfForNewEndEffectorBody = rotm2tform(rotationX(180));  
transfForNewEndEffectorBody(:,4) = [0.04; -0.195; 0; 1];  
newEndEffectorBody = rigidBody('dispenserEdge');  
setFixedTransform(newEndEffectorBody.Joint, transfForNewEndEffectorBody);  
glueDispenserBodyName = ur5_RBT.BodyNames{11};  
addBody(ur5_RBT,newEndEffectorBody,glueDispenserBodyName);
```

Biểu diễn mô hình robot bằng cách tạo các nodes tương tác cho các khâu và khớp trên RBT.

```
% Visualize the interactive rigid body tree at the home position ('q_home')
ur5_iRBT = interactiveRigidBodyTree(ur5_RBT);
q_home = [0 -90 0 -90 0 0]*pi/180;
rotate3d off;
view(145,25)
lightangle(20,-160)
axis([-1 1 -1 1 -0.5 1])
hold on
zlim([-0.5 1.5])
ur5_iRBT.ShowMarker = false;
ur5_iRBT.Configuration = q_home;
```

3.3.2. Xây dựng môi trường làm việc.

Tải đối tượng STL của đối tượng thao tác vào môi trường làm việc của robot trong Rigid Body Tree. Gắn chức năng gọi lại 'ButtonDownFcn' vào đối tượng thao tác để cho phép chọn điểm tham chiếu từ bề mặt đối tượng bằng cách nhấp vào nút chuột.

```
% Load STL object and get patch object
windshield = stlread('windshieldv3.stl');
orientationOfWindshield = 55; % In degree
locationOfWindshield = [0 0.4 0.15]; % With respect to global frame of the world [x y z]
windshieldPatch = patch(gca,'Faces',windshield.ConnectivityList,'Vertices',(windshield.Points + locationOfWindshield,'FaceColor',[0.8 0.8 0.9],'LineStyle','none'));

% Attaching the ButtonDownFcn callback
set(windshieldPatch,'ButtonDownFcn',{@exampleHelperSelectWaypoints});
```

Xác định điểm đến mong muốn khi thực hiện công việc cho robot bằng cách click chuột: Chọn điểm tham chiếu trên đối tượng trước khi thực hiện phần này. Sử dụng nút chuột để chọn điểm tham chiếu mong muốn. Có thể nhận thấy rằng sau mỗi lần lựa chọn thành công, trên thân kính chắn gió sẽ xuất hiện một điểm đánh dấu màu xanh lá cây. Sau khi chọn các điểm đích đến, ta xác định tọa độ của chúng bằng đoạn code dưới đây:

```
% Call the helper function "exampleHelperSelectWaypoints" to get selected
% waypoints data
selectedWaypoints = exampleHelperSelectWaypoints();

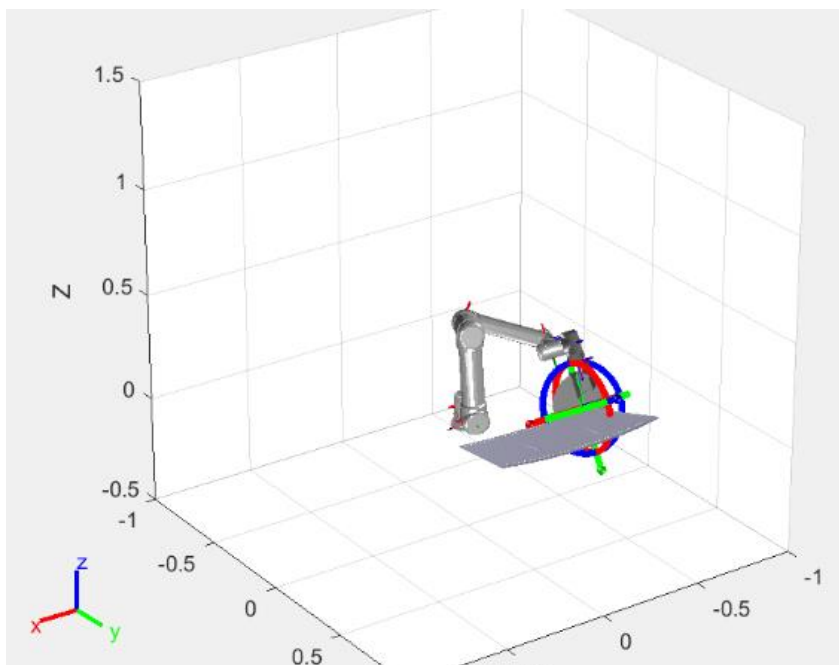
% Consider default waypoints set for further calculations if no waypoints are selected
% in the previous section
if isempty(selectedWaypoints)
    selectedWaypoints = [ -0.3186    0.4080    0.1514; -0.3350    0.4932    0.1720; -0.3653    0.6683    0.2139; -0.3817    0.7500    0.2338; -0.3948    0.8153    0.2397; -0.2397    0.8329    0.2407; -0.1565    0.8346    0.2368; -0.0603    0.8419    0.0311; 0.8442    0.2358; 0.1307    0.8383    0.2366; 0.3935    0.8144    0.2497; 0.3786    0.7400    0.2312; 0.3664    0.6682    0.2141; 0.3538    0.5981    0.1977; 0.3207    0.4120    0.1525; 0.2180    0.4132    0.1473; 0.1021    0.4186    0.1441; -0.0568    0.4185    0.1441; -0.1718    0.4180    0.1466; -0.3128    0.4086    0.1441];
end
```

Nếu chạy chương trình với quỹ đạo mặc định của robot trong quá trình khảo sát, ta không cần bước chọn điểm đích trên. Và sau khi chạy chương trình lấy tọa độ điểm mong muốn, hàm “ButtonDownFcn” bị ngắt hoạt động để cố định các điểm đã thiết lập, tránh trường hợp người sử dụng lựa chọn nhầm trong khi thao tác.

```
set(windshieldPatch,'ButtonDownFcn',[]);
```

Xác định quỹ đạo chuyển động bằng phương trình động học ngược.

```
ik = inverseKinematics('RigidBodyTree',ur5_RBT);  
% Disable random restarts  
ik.SolverParameters.AllowRandomRestart = false;  
ikWeights = [1 1 1 1 1 1];  
  
% Set robot position to first waypoint for orientation setting  
firstWaypoint = selectedWaypoints(1,:);  
orient = [0 0 -pi/3];  
tgtPose = trvec2tform(firstWaypoint) * eul2tform(orient); %target pose  
config = ik('dispenserEdge',double(tgtPose),ikWeights,'q_home');  
ur5_iRBT.Configuration = config;  
ur5_iRBT.ShowMarker = true;  
rotate3d off;
```



Hình 3.2. Xác định tư thế của robot tại điểm cần đi đến.

Đặt hướng của từng điểm tham chiếu mong muốn

Điều chỉnh hướng cho các điểm tham chiếu mong muốn bằng các phím trên bàn phím. Dưới đây là chi tiết về các phím gọi lại bàn phím kèm theo. Trước khi sử dụng nút bàn phím này, hãy đảm bảo rằng không có công cụ của sổ hình nào được chọn.

- Mũi tên lên / xuống: Để thay đổi hướng của điểm tác động cuối cobot về trục X
- Mũi tên phải / trái: Để thay đổi hướng của công cụ tác động cuối cobot về trục Y

- C: Để lưu dữ liệu định hướng cho điểm tham chiếu hiện tại
- N: Để di chuyển đến điểm tham chiếu tiếp theo từ điểm tham chiếu hiện tại
- Để bỏ qua điểm tham chiếu hiện tại và không muốn xem xét nó trong tính toán quỹ đạo thì chỉ cần nhấn nút 'N' mà không lưu dữ liệu định hướng.

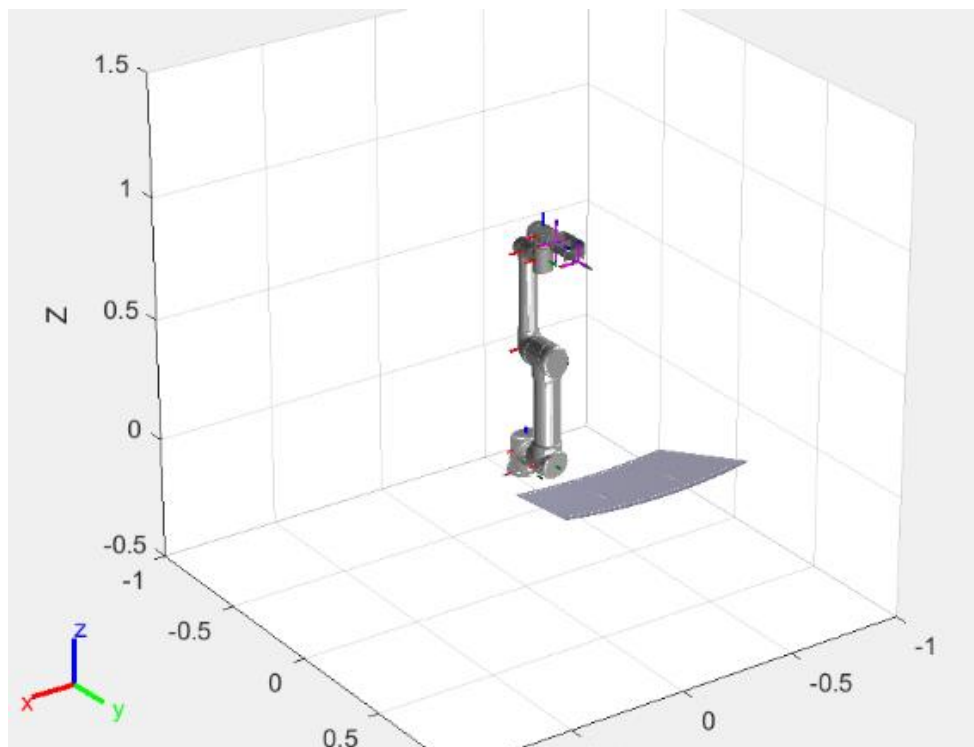
Xác định dữ liệu về vị trí điểm cuối và hướng di chuyển của robot: Nhận dữ liệu định hướng mong muốn cho việc lập kế hoạch chuyển động. Nếu hướng không được đặt cho các điểm tham chiếu mong muốn thì nó sẽ sử dụng hướng mặc định như được đề cập ở trước.

Thiết lập vị trí ban đầu:

```
% remove the keyboard callback
set(gcf,'KeyPressFcn',[]);

% Move cobot to the home position
ur5_iRBT.Configuration = q_home;

% Disable the marker from the tool body
ur5_iRBT.ShowMarker = false;
```



Hình 3.3. Vị trí ban đầu của robot.

Thiết lập tọa độ điểm đích:

```
% Get the desired waypoint and orientation data
[finalWaypoints, finalOrientations]= exampleHelperGetFinalWaypointData();

% Consider the default waypoint and orientation data if no orientation
% setting has been done for waypoints
if isempty(finalOrientations) && isempty(finalWaypoints)
    finalWaypoints = selectedWaypoints;
    finalOrientations = repmat([0 0 -pi/3],size(finalWaypoints,1),1);
end
```

3.3.3. Lập quỹ đạo chuyển động cho robot.

Thuật toán lập kế hoạch quỹ đạo không gian nhiệm vụ được sử dụng trong ví dụ này lấy đối số đầu vào là vận tốc TCP do người dùng xác định và độ phân giải thời gian để tính toán quỹ đạo. Đây là các bước cấp cao được thực hiện để tạo ra quỹ đạo mong muốn.

- Nhận các điểm tham chiếu và mảng định hướng mong muốn với tốc độ và độ phân giải thời gian TCP mong muốn.
- xem xét đoạn đường đi giữa hai điểm tham chiếu liên tiếp.
- Tìm khoảng cách của đoạn đường dẫn.
- Tìm thời gian đi qua đoạn đường đang xem xét từ khoảng cách được tính toán và vận tốc TCP đầu vào.
- Nhận các phép biến đổi điểm trung gian bằng cách sử dụng “Transformtraj” sử dụng mảng thời gian quỹ đạo được tính toán, xem xét độ phân giải thời gian mà bạn đã xác định.
- Tính toán dữ liệu chung bằng cách sử dụng “inverseKinematics” tại mỗi điểm trung gian.
- Tính vận tốc và gia tốc của khớp bằng “minjerkpolytraj”.

Xác định quỹ đạo dịch chuyển:

```
% Desired tool center point (TCP) velocity
velOfTCPForTask = 0.15;

% Desire time step resolution
dtForTask = 0.02;

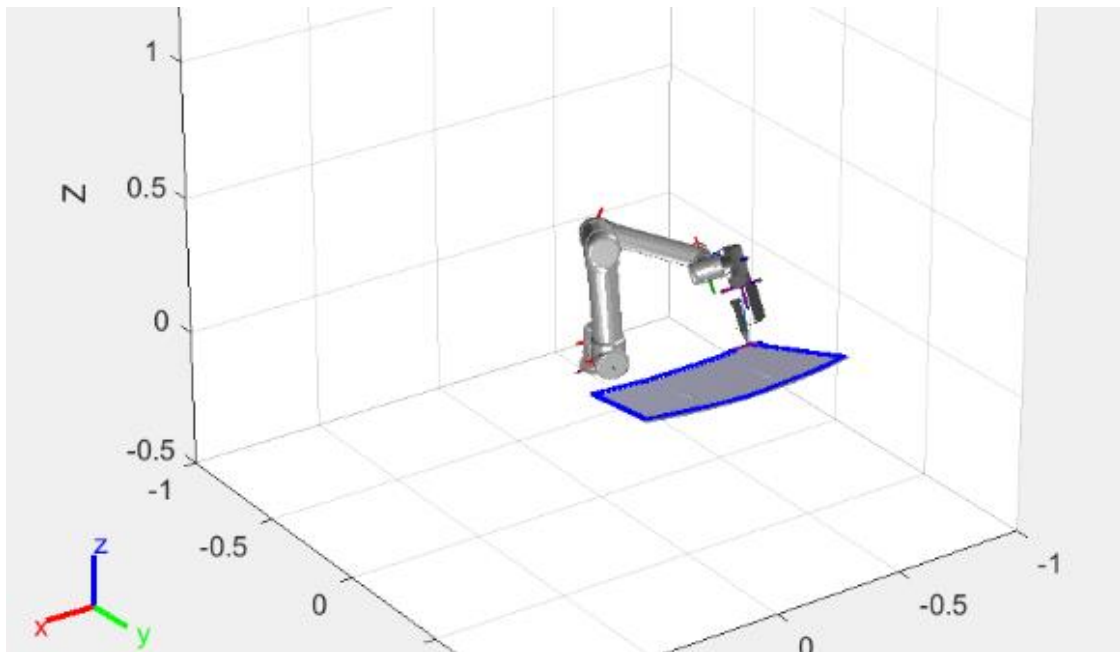
computedTrajForTask = exampleHelperURGenerateTrajectory(ur5_RBT,config',finalWaypoints,
trajTimes = 0:dtForTask:(size(computedTrajForTask.position,1)-1)*dtForTask;
```

Di chuyển robot lần lượt theo các bước trong quỹ đạo đã xác định ở bước trên:

```
% Move cobot to the first desired waypoint
ur5_iRBT.Configuration = computedTrajForTask.position(1,:);
```

3.3.4. Kết quả thực hiện.

Biểu diễn quỹ đạo mà robot thực hiện tại điểm thao tác cuối tại hình 3.4.



Hình 3.4. Quỹ đạo di chuyển của robot tại đường nét xanh.

Quá trình thực hiện biểu diễn quỹ đạo cho robot.

```
% TCP position in cartesian coordinates
postTCPForTask = computedTrajForTask.postTCP;

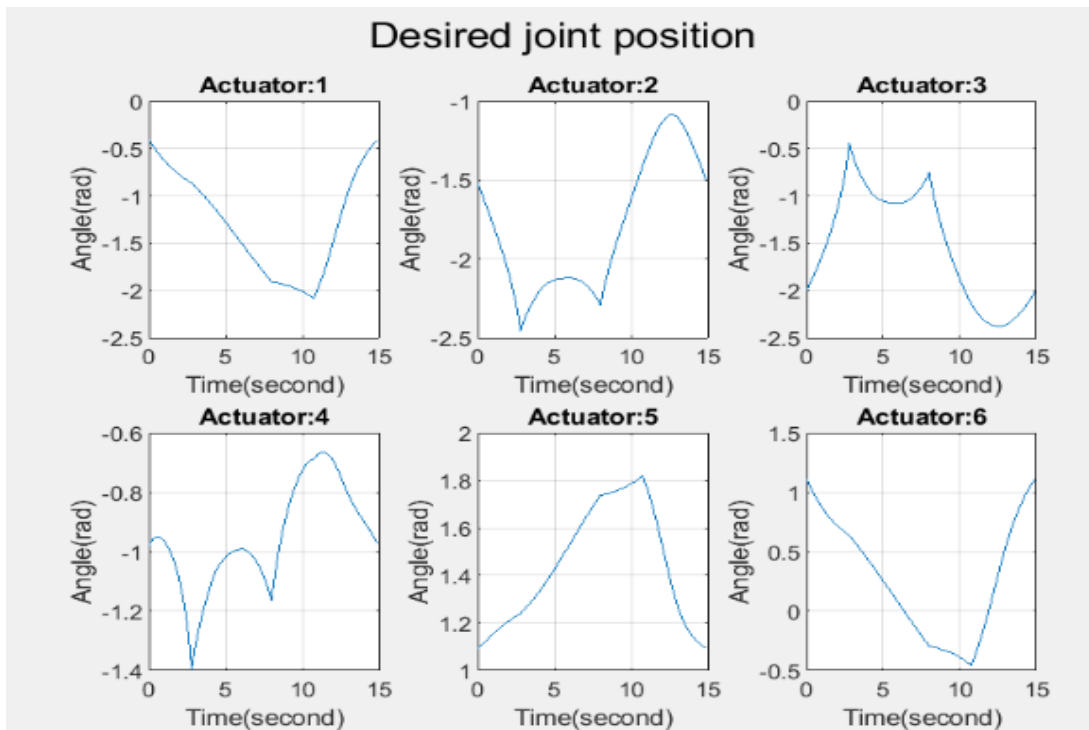
r = rateControl(1/dtForTask); % Rate to control the for loop execution
for idx = 1:size(computedTrajForTask.position,1)
    config = computedTrajForTask.position(idx,:);
    ur5_iRBT.Configuration = config';
    plot3(postTCPForTask(idx,1),postTCPForTask(idx,2),postTCPForTask(idx,3),'color','b','MarkerSize',10);
    waitfor(r);
end
```

Xác định các thông số hoạt động của robot trong quá trình di chuyển.

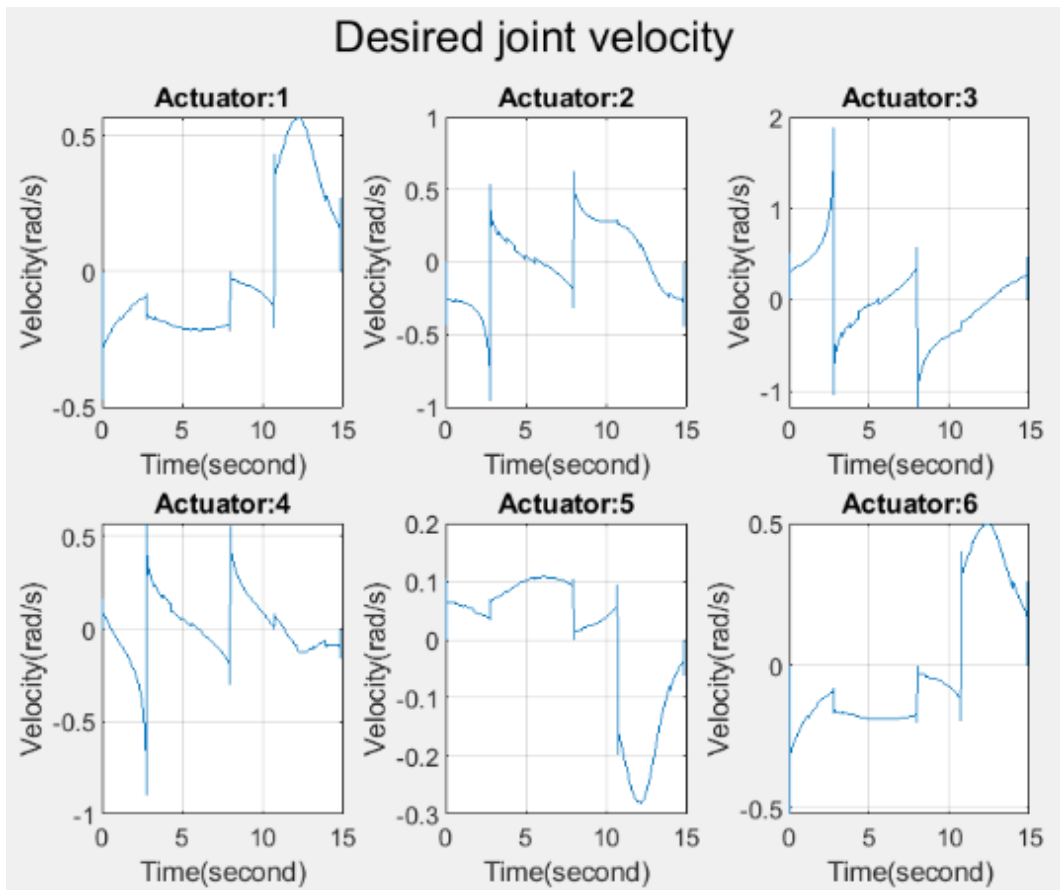
```
eeSpeed = zeros(length(computedTrajForTask.position),6);
for i=1:length(computedTrajForTask.position)
    % Calculate geometric jacobian
    jacobian = geometricJacobian(ur5_RBT,computedTrajForTask.position(i,:), 'dispenserE');

    % Calculate end-effector velocity
    eeSpeed(i,:) = (jacobian*computedTrajForTask.velocities(i,:))';
end

close(findobj('type','figure','name','Desired joint position'));
hFig1 = figure('Name','Desired joint position');
set(hFig1,'Visible','on')
sgtitle("Desired joint position")
for i=1:6
    subplot(2,3,i);
    plot(trajTimes,computedTrajForTask.position(:,i));
```



Hình 3.5. Biểu diễn giá trị tại các góc khớp của robot khi thực hiện quỹ đạo.
Biểu diễn giá trị vận tốc góc khớp trong quá trình di chuyển.



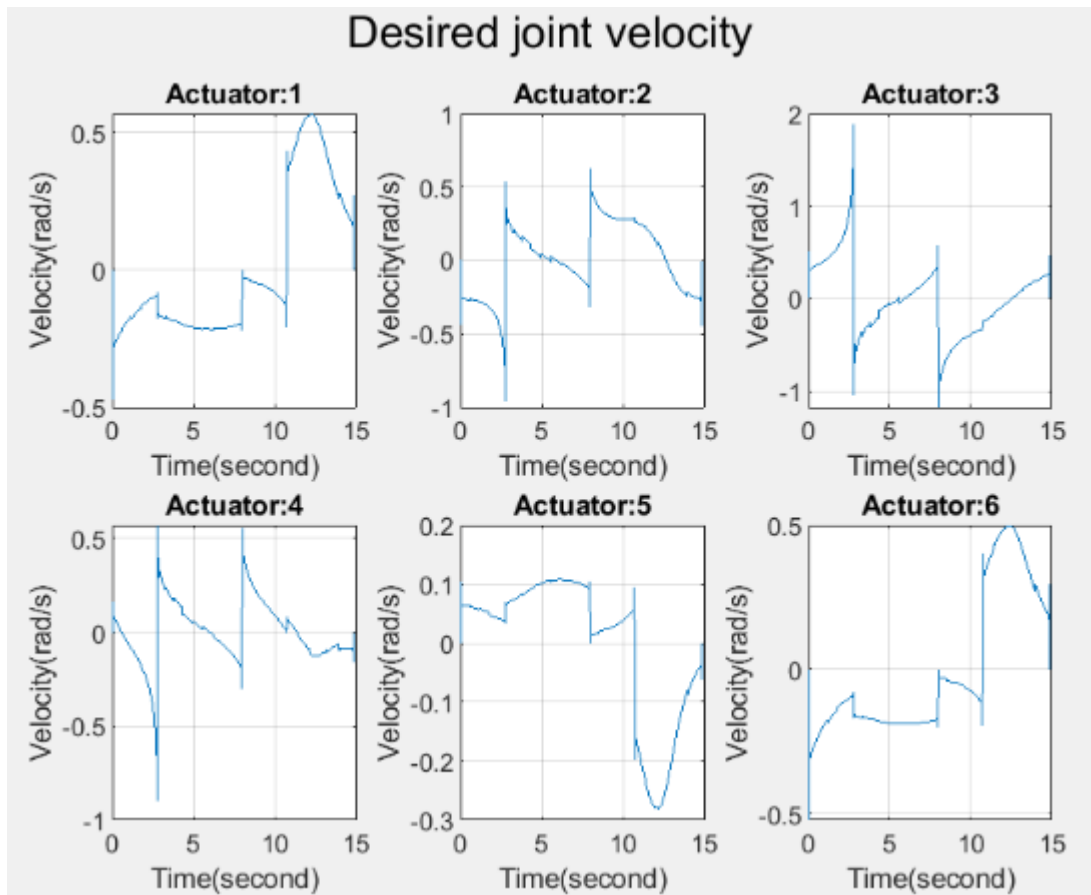
Hình 3.6. Giá trị vận tốc góc khớp trong quá trình di chuyển.

Chương trình thực hiện:

```
close(findobj('type','figure','name','Desired joint velocity'));
hFig2 = figure('Name','Desired joint velocity');
set(hFig2,'Visible','on')
sgtitle("Desired joint velocity")
for i=1:6
    subplot(2,3,i);
    plot(trajTimes,computedTrajForTask.velocities(:,i));
    titleString = "Actuator:" + num2str(i);
    title(titleString);
```

Biểu diễn các giá trị gia tốc góc khớp trong quá trình di chuyển theo quỹ đạo hình chữ nhật nối các điểm mong muốn đã chọn cho robot trong đoạn code chương trình dưới đây:

```
close(findobj('type','figure','name','Desired joint acceleration'));
hFig3 = figure('Name','Desired joint acceleration');
set(hFig3,'Visible','on')
sgtitle("Desired joint acceleration")
for i=1:6
    subplot(2,3,i);
    plot(trajTimes,computedTrajForTask.acceleration(:,i));
    titleString = "Actuator:" + num2str(i);
```

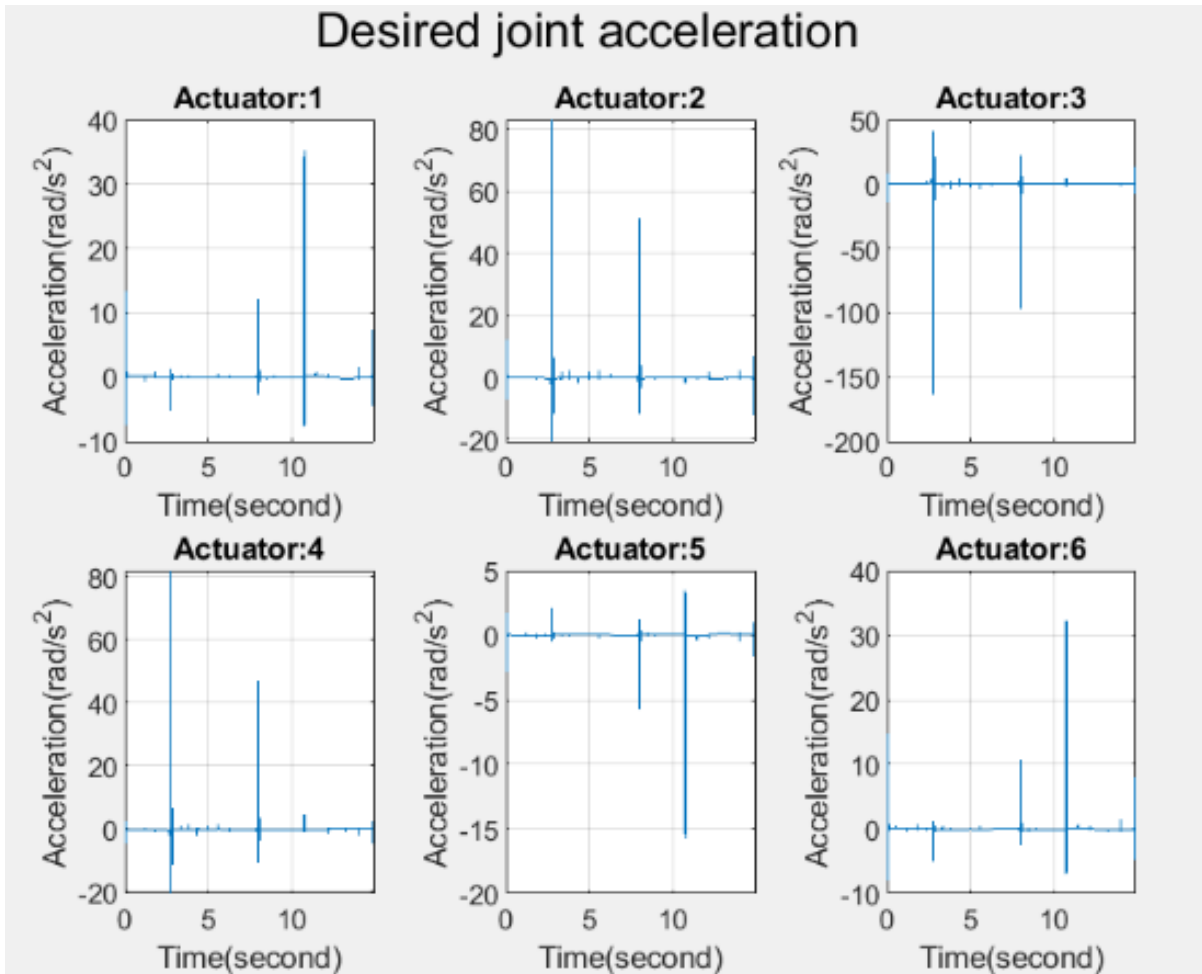


Hình 3.7. Gia tốc góc khớp của robot khi di chuyển.

Biểu diễn sai số gia tốc góc khớp tính lại.

```
close(findobj('type','figure','name','Desired joint acceleration'));
hFig3 = figure('Name','Desired joint acceleration');
set(hFig3,'Visible','on')
sgtitle("Desired joint acceleration")
for i=1:6
    subplot(2,3,i);

    plot(trajTimes,computedTrajForTask.acceleration(:,i));
    titleString = "Actuator:" + num2str (i);
    title(titleString);
    ylabel("Acceleration(rad/s^2)");
    xlabel("Time(second)");
    grid on;
    axis auto;
end
```



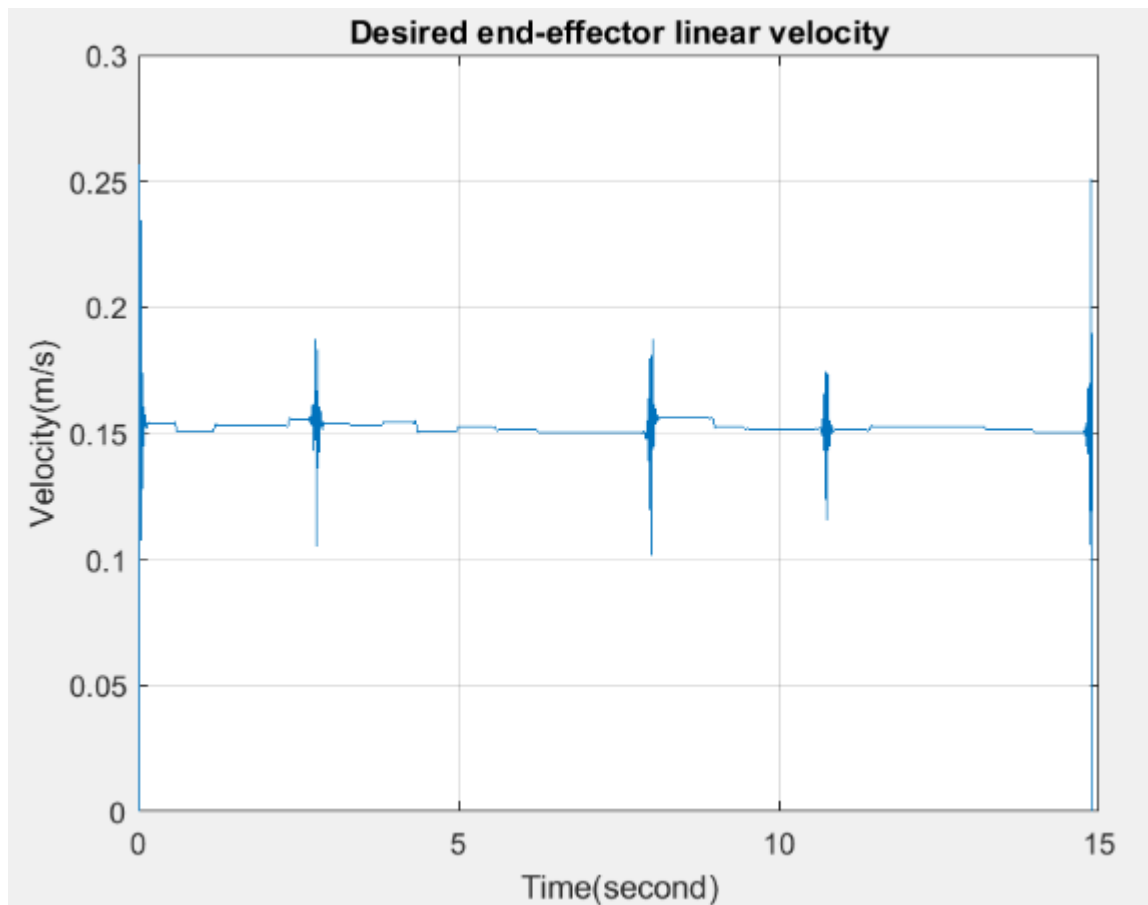
Hình 3.8. Biểu diễn sai số tính lại của giá trị gia tốc góc khớp.

Biểu diễn giá trị sai số tính lại của vận tốc tại điểm thao tác cuối trong không gian làm việc được trình bày trong đoạn mã dưới đây.

```

close(findobj('type','figure','name','Desired end-effector linear velocity'));
hFig4 = figure('Name','Desired end-effector linear velocity');
set(hFig4,'Visible','on')
plot(trajTimes,vecnorm(eeSpeed(:,4:6)'));
title("Desired end-effector linear velocity")
ylabel("Velocity(m/s)");
xlabel("Time(second)");
grid on;
axis auto;

```



Hình 3.9. Biểu diễn sai số tính lại của giá trị vận tốc điểm thao tác cuối.

CHƯƠNG 4. LẬP KẾ HOẠCH CHUYỂN ĐỘNG CHO HỆ THỐNG ĐA CÁNH TAY ROBOT BẰNG PHƯƠNG PHÁP ĐIỀU KHIỂN PHÂN TÁN.

4.1. Tiếp cận vấn đề.

4.1.1. Thách thức trong điều khiển hệ đa robot

Rất nhiều quá trình điều khiển thực hiện công việc phức tạp có thể được chia thành những bộ nhiệm vụ nhỏ phân bố cho nhiều robot khác nhau thực hiện song song trong cùng một không gian làm việc. Tuy nhiên, để điều khiển điều hướng lên kế hoạch chuyển động cho hệ đa cánh tay robot vẫn đang là một thách thức cho giới nghiên cứu trong lĩnh vực điều khiển bởi bài toán điều khiển với số lượng bậc tự do của hệ đa cánh tay rất lớn trong một không gian làm việc nhỏ nguy cơ xảy ra va chạm rất cao.

Với phương pháp lên kế hoạch chuyển động truyền thống như trong các nghiên cứu^{3 4 5 6 7 8} có lợi thế về thông số cho mọi mô hình thiết kế kế hoạch chuyển động của robot có sẵn từ trước. Tuy nhiên những phương pháp trên không thể thực hiện hiệu quả với số lượng cánh tay robot lớn bởi hệ thống điều khiển lúc này trở thành một hệ điều khiển cổ chai khi có quá nhiều thông tin đầu vào cần kiểm soát. Vấn đề về kích thước mô hình này giới hạn rất nhiều trong quá trình áp dụng mô hình hệ đa cánh tay robot bởi số lượng cánh tay hoạt động trong một không gian làm việc chung nhỏ gây nguy cơ va chạm lớn giữa các tác tử di động trong môi trường động lực học.

Một trong những phương pháp tiếp cận khác trong vấn đề lập kế hoạch chuyển động cho robot là phương pháp điều khiển phân tán, tiếp cận hệ đa cánh tay robot như một hệ thống đa tác tử. Tại đây, mỗi cánh tay được điều khiển bởi một tác tử nhận tín hiệu đầu vào thông tin quan sát môi trường diễn ra tất cả các trạng thái cả tác tử khác và tính toán quá trình chuyển động chỉ cho bản thân nó. Thông thường, phương pháp điều khiển phân tán hoạt động tốt với môi trường số lượng tác tử thay đổi, nhưng để thiết kế một bộ điều khiển cho công việc phức tạp như điều khiển hệ thống đa cánh tay robot vẫn đang là một thách thức lớn. Chỉ qua quá trình quan sát trạng thái của tác tử khác quan sát được trong môi trường, phương án lập kế hoạch chuyển động phân tán phải đáp ứng được điều kiện di chuyển độc lập không va chạm giữa các tác tử trong hệ và di chuyển đến được vị trí yêu cầu đặt được. Một phương pháp lập kế hoạch chuyển động tốt cho hệ thống đa cánh tay robot cần có những đặc điểm sau:

- Có thể thay đổi số lượng robot: thời gian chạy chương trình có thể tăng hợp lý khi tăng số lượng cánh tay hoạt động trong hệ.
- Có tính hợp tác: mỗi cánh tay có thể di chuyển đến được mục tiêu của nó và đảm bảo không va chạm với vật cản động là những cánh tay khác trong quá trình di chuyển của chúng trong cả hệ.
- Điều khiển vòng lặp kín: Mô hình lập kế hoạch chuyển động cho hệ cần phản ứng nhanh chóng với tác động của ngoại vật động trong môi trường và tác động sinh ra từ quá trình di chuyển của cánh tay. Do đó, để tránh những bước tính toán thừa thãi, một

mô hình lập kế hoạch chuyển động bằng vòng lặp kín chỉ tính toán trước một bước di chuyển cho cánh tay dịch đến đích một bước một thay vì tính toán toàn bộ quỹ đạo chuyển động để di chuyển đến điểm cuối.

- **Linh hoạt:** Mô hình xây dựng cần phù hợp chạy được với bất kì số lượng cánh tay robot nào đưa vào hệ thống mà không cần quá trình tính toán hay xây dựng lại bài toán tiếp cận từ đầu. Đặc điểm trên là điều tất yếu và quan trọng nhất để áp dụng vào hệ đa cánh tay robot có mô hình động lực học thay đổi mỗi lần thêm robot vào hệ, hay xóa robot khỏi hệ, hay khi robot di động trên môi trường.

4.1.2. Một số nghiên cứu liên quan.

Sampling-based Motion Planning. Các thuật toán lập kế hoạch chuyển động dựa trên lấy mẫu (SMP) chẳng hạn như Cây khám phá ngẫu nhiên nhanh (RRT)^{1 9} [1, 8], Cây khám phá ngẫu nhiên nhanh tối ưu (RRT*)¹⁰ [9], RRT có hướng dẫn* (P-RRT*)¹¹ [10] và các biến thể hai chiều của chúng^{12 13} [11, 12] từ lâu đã trở thành giải pháp phù hợp cho việc lập kế hoạch chuyển động chung, do tính hiệu quả của chúng trong thực tế và đảm bảo tính đầy đủ về mặt lý thuyết. Tuy nhiên, chúng gặp khó khăn trong việc mở rộng quy mô trong các mô hình đa chiều có ứng dụng trong thế giới thực vì thời gian chạy của chúng theo cấp số nhân theo số DoF của hệ thống, tức là tổng số bậc tự do của từng rô-bốt trong các hệ thống nhiều rô-bốt. Tuy nhiên, một giải pháp thay thế nhanh hơn cho RRT trong thời gian chạy là Bản đồ đường xác suất - Probabilistic Road Maps (PRM)¹⁴ [13], tuy nhiên, việc tạo ra các lộ trình đủ dày đặc cho các hệ thống nhiều robot lớn là tốn kém và không linh hoạt.

Learning-based Motion Planning. Lập kế hoạch chuyển động dựa trên học tập. Nhiều công trình gần đây đã đề xuất các phương pháp tiếp cận dựa trên học tập, đánh đổi giai đoạn đào tạo ngoại tuyến rất dài để thực hiện nhanh hơn trong thời gian thực. Một số công việc này đã cải thiện tốc độ của SMP bằng cách làm sai lệch phân phối lấy mẫu của các nodes mới^{15 16 17} [14, 15, 16], dự đoán trực tiếp hành động tiếp theo¹⁸ [17], sử dụng kinh nghiệm trong quá khứ với mô-đun bảng tra cứu bổ sung¹⁹ [18] hoặc giảm không gian tìm kiếm thông qua rời rạc hóa^{5 6 20} [5, 6, 19]. Những người khác đã khám phá ứng dụng của học tăng cường vào lập kế hoạch chuyển động^{21 22 23 24 25} [20, 21, 22, 23, 24]. Ví dụ: Jurgenson và Tamar²⁶ [25] đã kết hợp chính sách xác định gradient sâu Deep Deterministic Policy Gradient²⁷ [26] với các minh chứng của chuyên gia từ RRT* để lập kế hoạch chuyển động phẳng của cánh tay robot 4-DoF. Tuy nhiên, vì các phương pháp này không giải thích rõ ràng cho nhiều robot nên chúng chỉ có thể hoạt động theo kiểu tập trung và do đó không thể mở rộng được.

Centralized Multi-robot Motion Planning. Lập kế hoạch chuyển động đa robot tập trung. Bài toán lập kế hoạch chuyển động nhiều cánh tay đã được nghiên cứu trong các ứng dụng như di chuyển đồ vật²⁸ [27] và sắp xếp đồ vật²⁹ [28]. Mirrazavi Salehian và cộng sự. [4] đã đề xuất một bộ điều khiển nhiều cánh tay vòng kín nhanh có khả năng lập kế hoạch lại ở tần số 500Hz cho các hệ thống cánh tay kép, nhưng dựa vào bộ giải Động học nghịch đảo (IK) tập trung, mà các ràng buộc phải được xác định lại từ đầu nếu các cánh tay được thêm vào,

loại bỏ từ hoặc di chuyển xung quanh không gian làm việc. Berg và cộng sự. [3] đã khám phá một phương pháp lập kế hoạch ưu tiên cho các hệ thống nhiều robot, nhưng được sử dụng³⁰[29] để lập kế hoạch chuyển động là vòng lặp mở và yêu cầu tính toán lộ trình dài cho từng không gian làm việc cụ thể. Quan trọng nhất, các phương pháp này có khả năng mở rộng kém vì chúng có các thành phần tập trung có thể đóng vai trò là điểm cố định trong hệ thống.

Decentralized Multi-robot Motion Planning. Lập kế hoạch chuyển động đa robot phi tập trung. Để đạt được khả năng mở rộng hệ robot trong quá trình hoạt động, mô hình xác định chuyển động nên thuộc điều khiển phân tán, trong đó mỗi robot có kế hoạch đạt được trạng thái mục tiêu của nó. Non-Holonomic Optimal Reciprocal Collision Avoidance (NH-ORCA)³¹ [30] là một cách tiếp cận phổ biến để lập kế hoạch nhiều robot phân tán và tránh va chạm, nhưng rất nhạy cảm với các tham số tinh chỉnh. Các công trình trước đây của MARL về lập kế hoạch chuyển động cho nhiều rô-bốt^{32 33}[31, 32] chỉ nghiên cứu quy hoạch không gian cấu hình chiều thấp, liên kết đơn của rô-bốt mặt đất và không hoạt động cho các hệ thống nhiều cánh tay được liên kết chặt chẽ của chúng tôi với các cánh tay rô-bốt 6-DoFs.

4.1.2. Xác định vấn đề.

Đề tài sử dụng mô hình học tăng cường đa tác tử - Multi-agent Reinforcement learning để học từ một chính sách chuyển động phân tán của robot. Chính sách này cung cấp một hàm điểm thưởng chung cho cả hệ thống khi tất cả cánh tay đi được đến vị trí đích của nó để ràng buộc các robot cần hợp tác cùng nhau trong quá trình di chuyển. Kết quả của chính sách trên ngăn vấn đề xác định điểm thưởng đi vào lối mòn trong quá trình khám phá của mô hình học tăng cường khi thuật toán khám phá ngẫu nhiên tất cả các trường hợp để tìm ra chính sách đưa ra hành động tốt nhất. Cũng bởi lý do trên, những chính sách sao chép hành vi từ mẫu dữ liệu chuyển động của robot (ví dụ như Cây Phân khối ngẫu nhiên hai hướng – Bidirectional Rapidly-exploring Random Tree, BiRRT) cũng hoạt động không tốt bởi chương trình không quan sát ghi nhận đủ mẫu dữ liệu xấu khi robot vận hành tệ để học được ranh giới giữa các không gian hoạt động của robot và xác định biên va chạm trong môi trường, một điều kiện cần thiết cho chính sách thiết lập hoạt động trong những trạng thái cận va chạm.

Để giải quyết những thách thức trên, chúng tôi đề xuất cung cấp các mô phạm của robot (bằng điều khiển tập trung với BiRRT) trên các nhiệm vụ thất bại, và trên các lần thất bại của chính sách, hướng tiếp cận trên có những ưu điểm sau. Thứ nhất, bằng cách đối chiếu trực tiếp quỹ đạo thất bại và thành công của cùng một nhiệm vụ, chính sách có thể loại bỏ mức tối thiểu cục bộ trong khi vẫn xét đủ số trường hợp không thành công. Thứ hai, khi chính sách được cải thiện trong quá trình đào tạo, nó đòi hỏi ít sự mô phạm ban đầu hơn và bắt đầu thu thập các quỹ đạo thành công thông qua việc tự khám phá, điều này cho phép chính sách cải thiện ngoài dữ liệu điều khiển của chuyên gia. Cuối cùng, bằng cách xấp xỉ một mạng thần kinh nhanh, hệ thống duy trì tốc độ suy luận nhanh tại thời điểm thử nghiệm, cho phép nó phản ứng nhanh với các động thái chưa xác định của mục tiêu và lập kế hoạch lại. Chính sách lập kế hoạch chuyển động của chúng tôi đạt được tỷ lệ thành công cao trong các nhiệm vụ lập

kế hoạch chuyển động tĩnh và động đa nhánh được kết hợp chặt chẽ, điều mà cả phương pháp tiếp cận chỉ dựa trên lấy mẫu và dựa trên huấn luyện đều không thể đạt được một mình.

Đề xuất của chúng tôi là cách tiếp cận MARL để đào tạo một bộ lập kế hoạch chuyển động đa nhánh phân tán, vòng kín, hợp tác, có quy mô tuyến tính với số lượng nhánh trong hệ thống và có thể lập kế hoạch ở tốc độ 920Hz trên một luồng CPU. Chính sách của chúng tôi sử dụng mô-đun Bộ nhớ ngắn hạn dài (LSTM)³⁴ [7] để mã hóa số lượng nhánh có thể thay đổi, cho phép nó linh hoạt tùy theo quy mô hệ robot. Mặc dù chỉ được đào tạo về các nhiệm vụ lập kế hoạch chuyển động tĩnh 1-4 cánh tay, cách tiếp cận của chúng tôi khái quát hóa các nhiệm vụ lập kế hoạch chuyển động tĩnh và động 5-10 cánh tay trong thời gian thử nghiệm.

4.2. Học tăng cường

4.2.1. Khái quát về học tăng cường

Học tăng cường (RL) là lĩnh vực nghiên cứu trong học máy giải quyết vấn đề làm thế nào để một đối tác tự điều chỉnh hành vi của mình qua các tương tác với môi trường để tối đa hóa một số khoản thưởng kỳ vọng. Các yếu tố cơ bản của RL bao gồm:

- Agent (Người học/AI học tập): thực thể đưa ra quyết định.
- Environment (Môi trường): nơi agent tương tác và nhận thưởng.
- Action (Hành động): những quyết định mà agent có thể lựa chọn ở mỗi trạng thái.
- State (Trạng thái): mô tả về môi trường mà agent quan sát được.
- Reward (Thưởng): tín hiệu ngắn hạn mà agent nhận được sau mỗi hành động.
- Policy (Chính sách): chiến lược hành động mà agent theo dõi.
- Value Function (Hàm giá trị): dự đoán tiềm năng thưởng kỳ vọng cho từng trạng thái hoặc cặp trạng thái-hành động.

Mục tiêu của RL là tối ưu chính sách để tối đa hóa tổng thưởng theo thời gian:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (4.1)$$

trong đó R_{t+k+1} là phần thưởng sau hành động tại thời điểm $t+k$ và γ là hệ số giảm độ quan trọng của điểm thưởng trong tương lai so với điểm thưởng tức thì.

Có hai loại phương pháp chính trong RL:

- Model-based: Phương pháp dựa trên mô hình môi trường.
- Model-free: Phương pháp không dựa vào mô hình môi trường và trực tiếp ước lượng giá trị hoặc chính sách.

Trong đó, phương pháp Model-free có thể được thể hiện qua hai kỹ thuật chính:

- *Monte Carlo Methods*: Tính giá trị cuối cùng dựa trên kết quả của toàn bộ tập trả phạt, không cần dự đoán sau mỗi bước, phù hợp với các tác vụ với đường đi rõ ràng từ bắt đầu tới kết thúc.
- *Temporal Difference Learning (TD)*: Kết hợp cả cách học của Monte Carlo và

Dynamic Programming, cập nhật giá trị dựa trên ước tính tạm thời chứ không cần toàn bộ chuỗi kết quả.

Đánh giá tối ưu trong học tăng cường được cài đặt và điều chỉnh *reward function* quyết định động lực và cách tiếp cận mà agent sẽ thực hiện. Để tối ưu chính sách, RL thường sử dụng các phương pháp như Policy Gradient, nơi agent cố gắng cập nhật tham số của chính sách $\pi(a|s)$ để tối đa hóa kỳ vọng thưởng thông qua đạo hàm hàm giá trị. Cách cập nhật và đánh giá cải thiện chính sách đưa ra hành động cho tác tử sẽ được nói kĩ hơn ở phần sau về phương pháp Soft Actor-Critic.

4.2.2. Học Tăng cường Đa tác tử.

Học tăng cường đa tác tử (MARL) là một lĩnh vực con của học tăng cường. Nó tập trung vào nghiên cứu hành vi của nhiều tác nhân học tập cùng tồn tại trong một môi trường dùng chung³⁵ Mỗi tác tử được thúc đẩy bởi phần thưởng riêng của nó và thực hiện các hành động để nâng cao lợi ích của chính mình; trong một số môi trường, những lợi ích này trái ngược với lợi ích của các tác nhân khác, dẫn đến động lực nhóm phức tạp.

Học tăng cường đa tác tử có liên quan chặt chẽ đến lý thuyết trò chơi và đặc biệt là các trò chơi lặp lại, cũng như các hệ thống đa tác tử. Nghiên cứu của nó kết hợp việc theo đuổi việc tìm kiếm các thuật toán lý tưởng nhằm tối đa hóa phần thưởng với một tập hợp các khái niệm mang tính xã hội học hơn. Trong khi nghiên cứu về học tăng cường đa tác nhân liên quan đến việc tìm ra thuật toán đạt số điểm lớn nhất cho một tác nhân, thì nghiên cứu về học tăng cường đa tác nhân đánh giá và định lượng các số liệu xã hội, chẳng hạn như hợp tác³⁶, tính tương hỗ³⁷, sự công bằng³⁸, ảnh hưởng nhóm³⁹...

Tương tự như học tăng cường một tác nhân, học tăng cường đa tác nhân được mô hình hóa dưới dạng một số dạng của quy trình quyết định Markov (MDP). Ví dụ:

Cho một tập S các trạng thái trong môi trường.

Một tập $A_i : i \in \{1, \dots, N\}$ là tập trạng thái cho tất cả các tác tử hoạt động.

Xác suất chuyển từ trạng thái s sang trạng thái s' tại chuỗi hành động $\vec{a}$ là

$$P_{\vec{a}}(s, s') = \Pr(s_{t+1} = s' \mid s_t = s, \vec{a}_t = \vec{a}) \quad (4.2)$$

$\vec{R}_{\vec{a}}(s, s')$ là phần thưởng nhóm tức thì tại thời điểm chuyển trạng thái với hành động $\vec{a}$

Đối với hệ học tăng cường đa tác tử, sẽ xảy ra hai trường hợp khi tác tử cùng hợp tác để tất cả tác tử đều có thể hoàn thành công việc của mình, trường hợp khác khi các tác tử cạnh tranh chỉ nhằm lấy được điểm thưởng của mình mà ngăn cản quá trình làm việc của tác tử khác. Ví dụ trong hệ đa robot của chúng ta, trường hợp hợp tác khi các tác tử di chuyển hài hòa hợp lý, không xảy ra va chạm; trường hợp cạnh tranh khi lộ trình di chuyển của ít nhất một cặp tác tử cắt nhau và cặp tác tử này không nhường đối phương và di chuyển va chạm nhau khi vận hành.

Khi nhiều tác tử hoạt động trong một môi trường chung, lợi ích của chúng có thể được liên kết hoặc không đồng bộ. MARL cho phép khám phá tất cả các sắp xếp khác nhau và cách chúng ảnh hưởng đến hành vi của các tác tử khác: Trong môi trường cạnh tranh thuần túy, phần thưởng của các tác tử hoàn toàn trái ngược nhau và do đó chúng đang thi đấu với nhau.

Bối cảnh hợp tác thuần túy là một thái cực khác, trong đó các đại lý nhận được phần thưởng giống hệt nhau và do đó họ chơi với nhau. Cài đặt tổng giá trị độ hỗn loạn bao gồm tất cả các tác tử kết hợp các yếu tố cả hợp tác và cạnh tranh.

4.3. Phương pháp Soft Actor-Critic

Phương pháp Actor-Critic là một kỹ thuật quan trọng trong học tăng cường (Reinforcement Learning - RL) và là hợp nhất của hai phương pháp chính: policy-based và value-based approaches. Trong khuôn khổ Actor-Critic, Actor và Critic đóng vai trò như hai phần tử chính thức hoạt động song song để tối ưu hóa cùng một mục tiêu cuối cùng là hành động tối ưu của Agent trong môi trường xác định.

Actor: đại diện cho mô hình chính sách (policy) $\pi(a|s; \theta_\pi)$ mà tại đó a là hành động được chọn bởi Actor trong trạng thái s và θ_π là tham số của mô hình chính sách. Actor có nhiệm vụ đưa ra quyết định và hành động dựa trên trạng thái hiện tại của môi trường. Actor hướng dẫn RL agent lựa chọn hành động bằng cách sử dụng chính sách (policy) Policy này có thể được biểu diễn dưới dạng một vector tỷ lệ xác suất chọn một hành động khi ở một trạng thái nhất định. Chính sách có thể được tối ưu hóa thông qua Policy Gradient Methods, nơi Actor học để thay đổi tham số θ_π của nó nhằm tăng khả năng đạt phần thưởng cao hơn.

Critic: đánh giá hành động do Actor thực hiện dựa theo hàm giá trị $V(s; \theta_V)$ hoặc $Q(s, a; \theta_Q)$ với θ_V, θ_Q là tham số của mô hình giá trị. Critic được dùng để ước lượng "độ tốt" của trạng thái s hoặc cặp trạng thái-hành động (s, a) . Hàm giá trị này giúp Critic đánh giá được hành động thông qua việc tính toán lợi ích dài hạn, thường được biểu diễn dưới hình thức giá trị kỳ vọng của phần thưởng tích lũy.

Mối quan hệ giữa hành động a và trạng thái s tại thời điểm t có thể được viết dưới dạng xác suất như sau:

$$p(a_t | s_t, \theta_\pi) = \pi(a_t | s_t; \theta_\pi) \quad (4.3)$$

với chính sách được lựa chọn bằng phân phối xác suất softmax

$$\pi(a_t | s_t; \theta_\pi) = \frac{e^{f(s_t, a_t; \theta_\pi)}}{\sum_{a'} e^{f(s_t, a'; \theta_\pi)}} \quad (4.4)$$

Trong học tăng cường, ta thường sử dụng thuật ngữ "temporal difference error" (lỗi sai khác thời gian) để chỉ sự chênh lệch giữa giá trị ước lượng và giá trị thực tế:

$$\delta_t = R_{t+1} + \gamma V(s_{t+1}, \theta_V) - V(s_t; \theta_V) \quad (4.5)$$

trong đó R_{t+1} là phần thưởng ngay lập tức nhận được sau hành động a_t , $\gamma \in (0, 1)$ là hệ số giảm phần thưởng tương lai, $V(s_t; \theta_V)$ là giá trị ước lượng cho trạng thái s_t

Mục tiêu hàng đầu của quá trình huấn luyện Actor-Critic là tối ưu hóa tham số của cả hai mô hình để nhằm cải thiện chất lượng của các quyết định được đưa ra bởi Actor và feedback được cung cấp bởi Critic. Cải tiến này được thực hiện thông qua các thuật toán tối ưu hóa như gradient descent, với các gradients được tính toán từ lỗi sai khác thời gian và policy gradient.

Cập nhật tham số của Actor bằng Gradient Policy:

$$\begin{aligned}\nabla_{\theta_V} J(\theta_V) &= E_{s \sim p^\pi, a \sim \pi_\theta} \left[\nabla_{\theta_V} \log \pi(a | s; \theta_\pi) \delta_t \right] \\ \theta_\pi &\leftarrow \theta_\pi + \alpha_\pi \nabla_{\theta_V} J(\theta_V)\end{aligned}\tag{4.6}$$

Cập nhật tham số Critic bằng temporal difference learning

$$\begin{aligned}\nabla_{\theta_V} J(\theta_V) &= E_{s \sim p^\pi} \left[\nabla_{\theta_V} \left(R_{t+1} + \gamma V(s_{t+1}; \theta_V) - V(s_t; \theta_V) \right)^2 \right] \\ \theta_V &\leftarrow \theta_V - \alpha_V \nabla_{\theta_V} \left((R + \gamma V(s'; \theta_V) - V(s_t; \theta_V))^2 \right)\end{aligned}\tag{4.1}$$

Một số biến thể khác của phương pháp Soft Actor-Critic: Deep Deterministic Policy Gradient (DDPG), Asynchronous Advantage Actor-Critic (A3C), Advantage Actor-Critic (A2C), Trust Region Policy Optimization (TRPO) và Proximal Policy Optimization (PPO)

4.4. Phương án thực hiện.

4.4.1. Học tăng cường phân tán với mô phạm mẫu từ chuyên gia.

Chúng tôi đề xuất một phương pháp đào tạo một mô hình lập kế hoạch chuyển động nhiều cánh tay có sự hợp tác, điều khiển vòng kín, phân tán, có thể tính toán các kế hoạch chuyển động cho các hệ thống nhiều cánh tay với quy mô hệ thống thay đổi được để đạt được cả tư thế mục tiêu tĩnh và động. Chúng tôi sử dụng Học tập độc lập – Independent learning với phương pháp Soft Actor Critic, trong đó mỗi nhánh trong hệ thống nhiều nhánh được điều khiển bởi một tác tử trong hệ thống đa tác tử đồng nhất, hợp tác bằng cách sử dụng chính sách phân tán với trọng số chung cho tất cả robot trong mô hình. Chính sách lập kế hoạch chuyển động cấp thấp của chúng tôi có thể được kết hợp với bất kỳ mô hình nhiệm vụ cấp cao nào để đạt được tính không đồng nhất ở cấp độ nhiệm vụ cho bất kỳ quy mô hệ robot nào với điều kiện các cánh tay có cùng kiểu phản cứng với mô hình cánh tay robot UR5.

4.4.2. Khởi tạo nhiệm vụ vận hành cho chuyển động của hệ đa cánh tay.

Mỗi tác vụ lập kế hoạch chuyển động mà chúng tôi tạo ra có thể được sử dụng ở hai chế độ: (1) chế độ tĩnh, trong đó tư thế mục tiêu đứng yên và (2) chế độ động, trong đó tư thế mục tiêu di chuyển với tốc độ không đổi trong phạm vi 1 – 15 cm/s . Nhiệm vụ đào tạo và kiểm tra của chúng tôi được tạo bằng quy trình tương tự như sau.

Đầu tiên, đối với nhiệm vụ cho k cánh tay, ta lấy mẫu ngẫu nhiên k tư thế cơ sở. Sau đó, sau khi đặt cánh tay k vào k tư thế cơ sở, tạo ra tổ hợp 3 cấu hình khớp không va chạm $[q_1 \quad q_2 \quad q_3]$ bằng cách giải động học ngược cho 3 các vị trí tác động cuối mục tiêu ngẫu

nhiên, so với vị trí robot ban đầu. Kết quả đưa ra cấu hình thông số góc khớp không đồng nhất về hướng của bộ phận tác động cuối bởi không gian làm việc của hệ robot hẹp. Ở chế độ tĩnh, sử dụng q_1 làm cấu hình khớp ban đầu, q_2 để tính toán các tư thế tác động cuối mục tiêu bằng cách sử dụng động học thuận (FK) và không sử dụng q_3 . Trong chế độ động, sử dụng q_1 làm cấu hình khớp ban đầu, và nội suy giữa q_2 và q_3 trong suốt quá trình, tính toán nghiệm của phương trình động học thuận tại các góc khớp để có được các tư thế mục tiêu động hiện tại. Cách tiếp cận này tạo ra các nhiệm vụ có thể được sử dụng cho cả việc lập kế hoạch chuyển động tĩnh và động trong khi vẫn đảm bảo sự tồn tại của một giải pháp cho tất cả các nhiệm vụ.

Tập dữ liệu huấn luyện bao gồm 1.000.000 tác vụ ở chế độ tĩnh, được chia đều cho 1-4 nhánh và tập dữ liệu thử nghiệm bao gồm 30.000 tác vụ ở cả chế độ tĩnh và động, chia đều cho 1-10 nhánh.

4.4.3. Công thức huấn luyện đa tác tử trong hệ học tăng cường.

Chúng tôi xây dựng nhiệm vụ lập kế hoạch chuyển động nhiều cánh tay như một vấn đề học tăng cường đa tác tử đồng nhất, có sự hợp tác, trong đó mỗi cánh tay được điều khiển bởi một tham số của cùng một chính sách với trọng số được chia sẻ trên tất cả các cánh tay. Các tác nhân được đào tạo bằng cách sử dụng Học tập độc lập (IL) với Soft Actor Critic, trong đó các tác tử chia sẻ các thông số và kinh nghiệm.

Để đạt được sự hợp tác, chúng tôi sử dụng phần thưởng nhóm, phần thưởng này chỉ thưởng cho tất cả các cánh tay +1 khi tất cả các cánh tay đã đạt được mục tiêu tương ứng của vị trí điểm tác động cuối. Sử dụng phần thưởng của nhóm này và cho phép mỗi cánh tay quan sát trạng thái của các cánh tay khác trong không gian làm việc cục bộ của mình (Phần 4.3), các cánh tay được thúc đẩy và cung cấp đủ thông tin để không chặn các cánh tay khác khỏi mục tiêu của các cánh tay khác. Ngoài ra, các cánh tay nhận được phần thưởng +0,01 khi nó chạm tới mục tiêu riêng lẻ và bị phạt -0,05 khi va chạm. Chính sách trên được huấn luyện với thông số giảm điểm thưởng sau mỗi một quãng thời gian.

Sau mỗi một chu trình huấn luyện, một nhiệm vụ từ tập dữ liệu huấn luyện mẫu được lấy ra để xem xét. Một chu trình kết thúc khi có một trong những cánh tay của hệ xảy ra va chạm, hoặc khi tất cả các cánh tay đều di chuyển được đến điểm đích của nó, hoặc khi chu trình chạy quá 500 bước thời gian huấn luyện. Tất cả quá trình mà hệ cánh tay đã hoạt động tại chu trình trước được lưu lại để sử dụng trong việc tối ưu hóa chính sách hành động trong vòng lặp lần sau. Một nhiệm vụ được coi là thành công khi tất cả cánh tay di chuyển được đến điểm đích và không thất bại trong những điều kiện tiên quyết khác đã nêu phía trên.

4.4.4. Huấn luyện lại model tại những nhiệm vụ không thực hiện được.

Khi ta cung cấp hàm phần thưởng cho toàn hệ khuyến khích sự phối hợp nhóm giữa các robot, nhưng nó cũng dẫn đến phần thưởng riêng của từng robot thừa thớt hơn, vì vậy việc

khám phá ngẫu nhiên không thể đạt được đủ phản hồi tích cực. Để hướng dẫn việc khám phá ban đầu và giúp chính sách thoát khỏi mức tối thiểu cục bộ, chúng tôi cung cấp quỹ đạo bằng phương pháp điều khiển tập trung BiRRT một cách thành công cho chính sách tại các nhiệm vụ thất bại, giúp giải quyết vấn đề không đủ điểm thưởng cho tác tử xác định hành động trong khi vẫn cho phép chính sách rút kinh nghiệm từ những trải nghiệm thất bại. Ngoài ra, điều kiện của bài toán này xảy ra khi quỹ đạo của dữ liệu hướng dẫn thực hiện thành công chu trình chuyển động.

Trước khi đào tạo, hướng dẫn của BiRRT tập trung tính toán quỹ đạo dày đặc cho trong tập dữ liệu các nhiệm vụ để hoàn thành chúng. Trong quá trình đào tạo, nếu chính sách không thực hiện được một nhiệm vụ thì các điểm tham chiếu BiRRT được tính toán trước cho nhiệm vụ đó sẽ được chuyển đổi thành các hành động khớp delta của từng nhánh riêng lẻ và được thực thi. Những bộ dữ liệu thao tác thành công của người hướng dẫn robot được truyền vào hệ robot thực hiện và so sánh với chính sách cũ.

Số lượng điểm tham chiếu trong quỹ đạo dày đặc của BiRRT có thể vượt quá giới hạn bước thời gian, do đó, việc tính toán các hành động liên kết của mẫu hướng dẫn bằng cách trừ đi các bước không cho phép các cánh tay tiếp cận trực tiếp mục tiêu. Thay vào đó, chúng tôi coi quỹ đạo là một đường cong bao gồm các đoạn thẳng trong không gian khớp và giảm dần nó với hệ số dung sai góc là 0,01 radian để tối giản hóa quỹ đạo. Điều này tạo ra một chuỗi các hành động khớp tam giác gần đúng với quỹ đạo ban đầu của BiRRT với đủ chi tiết để tránh va chạm trong khi vẫn tiếp cận mục tiêu kịp thời.

Đối với các nhiệm vụ thất bại, việc so sánh những lần thực hiện thất bại có phần thưởng thấp của chính sách với trải nghiệm thành công có phần thưởng cao của mẫu mô phạm sẽ giúp cho chính sách trong giai đoạn thăm dò và hướng dẫn chính sách đó vượt ra khỏi mức tối thiểu cục bộ. Đối với các nhiệm vụ thành công, quỹ đạo mẫu không được cung cấp, tạo điều kiện cho chính sách có thể cải thiện ngoài dữ liệu ban đầu thông qua việc tự khám phá. Do đó, mức tối ưu thực tế của mẫu hướng dẫn không ảnh hưởng đến mức tối ưu cuối cùng của chính sách, bởi vì tác tử có thể tiếp tục cải tiến mức tối ưu của chúng thông qua các quỹ đạo thành công bên ngoài. Phương pháp Soft Actor Critic vì là thuật toán RL ngoài chính sách, học chính sách ngẫu nhiên tối đa hóa entropy cho không gian hành động liên tục để thực hiện cải thiện liên tục mô hình chuyển động của robot sau nhiều vòng lặp huấn luyện.

CHƯƠNG 5. KẾT QUẢ THỰC HIỆN

5.1. Phát biểu bài toán

Mục tiêu của nhiệm vụ lập kế hoạch chuyển động nhiều cánh tay là tìm ra quỹ đạo không va chạm của các điểm tham chiếu cho tất cả các cánh tay từ cấu hình khớp ban đầu của chúng để đạt được tư thế thao tác cuối mục tiêu. Giả sử i là một cánh tay với một điểm cố định là p_i^b và một không gian cấu hình khớp $C_i \subseteq \mathbb{R}^d : F_i \subseteq C_i$, trong đó $F_i \subseteq C_i$ là không gian tự do của

i – tập hợp tất cả các tư thế - hành động của i robot thực hiện không va có chạm. Chúng tôi nói rằng tôi đã đạt được tư thế thao tác cuối p_i^t mong muốn (từ nay trở đi được gọi là tư thế mục tiêu của hệ robot) nếu tư thế thao tác cuối hiện tại của cánh tay robot nằm trong khoảng cách p_i^e và một góc quay ε_p so với p_i^t . Bài toán lập kế hoạch chuyển động nhiều cánh tay cho N cánh tay robot được mô tả bằng các tư thế cơ bản của cánh tay, cấu hình giá trị khớp ban đầu và các tư thế mục tiêu. Đặt $F = F_1 \times F_2 \times \dots \times F_N$ là không gian trạng thái tự do tổng hợp của tất cả các cánh tay. Một giải pháp cho vấn đề lập kế hoạch chuyển động nhiều cánh tay là hàm liên tục $\sigma : [0,1] \rightarrow F$, sao cho $\sigma(0)$ là cấu hình khớp ban đầu tổng hợp và $\sigma(1)$ là cấu hình khi tất cả các cánh tay đều đạt đến tư thế mục tiêu.

5.2. Tiêu chí đánh giá.

Success Metric: Thanh đo mức thành công của mô hình - Chỉ số thành công. Nhiệm vụ thành công nếu có một bước thời gian trong tập trong đó tất cả các cánh tay đã đạt đến tư thế thao tác mục tiêu trong khoảng cách 2cm và góc quay 0,1radian. Ngoài ra, nếu bất kỳ nhánh nào trong hệ thống va chạm hoặc 500 bước thời gian huấn luyện đã trôi qua thì nhiệm vụ sẽ không thành công.



Hình 5.1. Các mức độ thực hiện nhiệm vụ khi độ rộng của không gian làm việc bị thu hẹp dần với mỗi trường hợp.

Task Difficulty: Độ khó của nhiệm vụ. Các nhiệm vụ được phân chia dựa trên độ khó, đây là thước đo mức độ gắn kết chặt chẽ của không gian làm việc và do đó, khả năng xảy ra va chạm giữa các cánh tay. Độ khó của một nhiệm vụ là % giao điểm tối đa của khối lượng không gian làm việc bán cầu của bất kỳ nhánh nào (được hiển thị trong Hình 5.1.) với tất cả khối lượng không gian làm việc bán cầu của nhánh khác. Việc lấy mức tối đa làm cho các nhiệm vụ có quy mô nhóm khác nhau có thể so sánh được (so với việc lấy tổng), trong khi vẫn được ưu tiên hơn là lấy giá trị trung bình vì một nhiệm vụ sẽ không thành công nếu bất kỳ nhánh nào trong hệ thống không thành công.

Testing Dataset: Bộ dữ liệu kiểm thử. Tập dữ liệu thử nghiệm bao gồm 30.000 nhiệm vụ với các tư thế cơ bản mới và tư thế tác động cuối mục tiêu, được chia đều cho 1-10 nhánh và được tạo theo cách tương tự như tập dữ liệu huấn luyện. Các nhiệm vụ tĩnh được phân chia dựa trên độ khó của chúng, trong đó các nhiệm vụ dễ, trung bình và khó có độ khó lần lượt là 0,0 – 0,35, 0,35 – 0,45 và 0,45 – 0,50. Thay vào đó, các nhiệm vụ động được phân chia dựa trên tốc độ của chúng, trong đó các nhiệm vụ chậm, trung bình và nhanh có độ khó lần lượt là 1 – 5 cm/s, 5 – 10 cm/s và 10 – 15 cm/s, với độ khó trung bình là 0,185 .

Triển khai BiRRT: Để tính toán tập dữ liệu trình diễn chuyên gia và thử nghiệm, việc triển khai BiRRT của chúng tôi sử dụng khoảng cách không gian khớp euclide làm hàm khoảng cách và được chạy trong 4000 lần lặp ở độ phân giải 0,05 radian. Đối với các chế độ tĩnh, BiRRT sử dụng cấu hình q_1 và mục tiêu ban đầu q_2 từ nhiệm vụ. Đối với các chế độ hoạt động, BiRRT sử dụng cấu hình khớp hiện tại của hệ thống nhiều nhánh làm cấu hình ban đầu và phép nội suy hiện tại giữa q_2 và q_3 làm cấu hình mục tiêu của nó. Hệ thống được phép thực hiện toàn bộ quỹ đạo BiRRT trước khi một tính toán mới bắt đầu với cấu hình mục tiêu và điểm bắt đầu mới nhất làm đầu vào. Đối với các tác vụ tĩnh, thời gian chạy của BiRRT được đo từ khi bắt đầu lấy mẫu cho đến khi tìm thấy quỹ đạo không va chạm cho tất cả các nhánh.

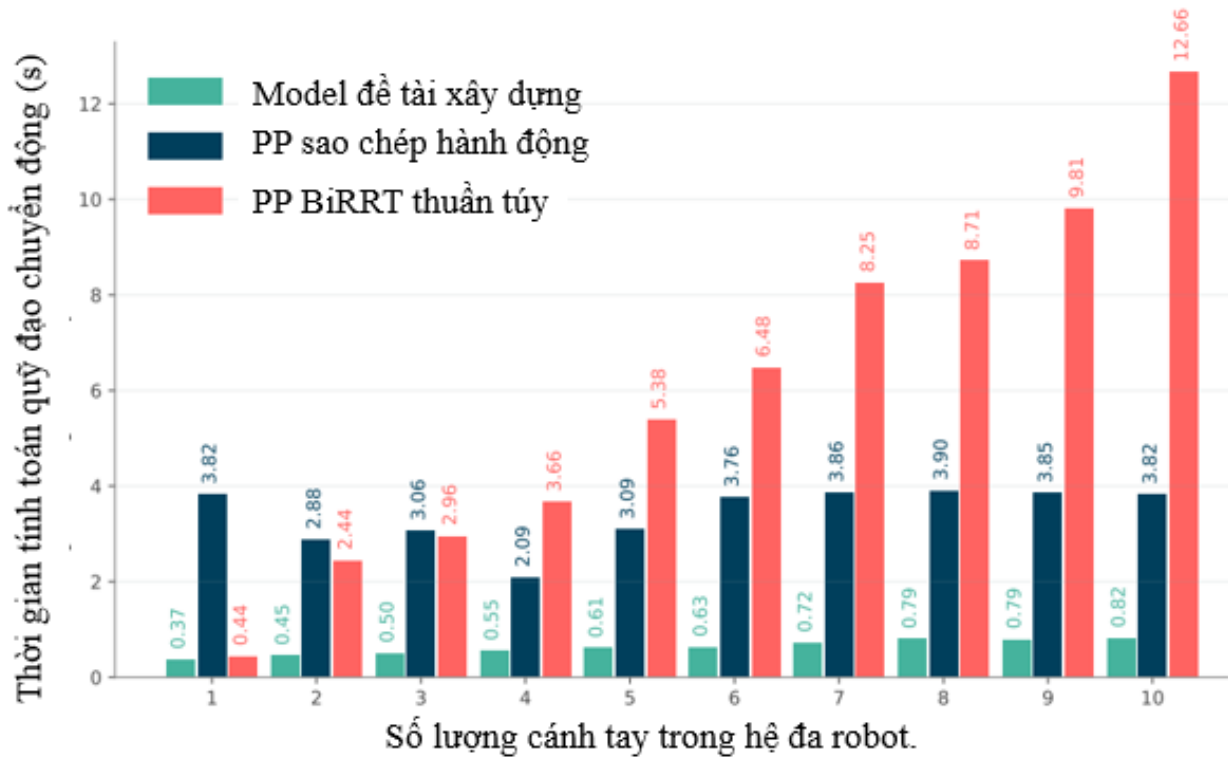
Phần cứng huấn luyện: huấn luyện mô hình trên hệ thống có CPU Intel i7-7820X 16 và GPU NVIDIA GTX 1080, việc đào tạo chính sách của chúng tôi mất 668,2 M bước thời gian (khoảng 14 ngày trên 700.000 tập) và tạo ra 1.000.000 điểm tham chiếu chuyên gia trong 2 ngày. Chúng tôi sử dụng PyBullet ⁴⁰ [36] làm môi trường đào tạo và thử nghiệm.

5.3. Kết quả và đánh giá.

5.3.1. Thời gian huấn luyện mô hình.

Đầu tiên, chúng tôi đánh giá khả năng mở rộng của thuật toán theo quy mô nhóm. Nhờ thiết kế phi tập trung, thời gian suy luận của mạng chính sách lập kế hoạch chuyển động của chúng tôi cho một lần chuyển tiếp là 1,09 ms (920Hz) với một luồng CPU duy nhất và không đổi so với quy mô nhóm. Tuy nhiên, để so sánh công bằng với thuật toán vòng lặp mở như BiRRT tập trung, chúng tôi vẽ sơ đồ thời gian chạy của phương pháp điều khiển tập trung BiRRT với tổng thời gian tính toán quỹ đạo trong phương pháp của chúng tôi, là tổng thời gian tính toán cho tất cả các bước cho đến khi đạt được mục tiêu, với một luồng CPU duy nhất trong Hình 5. Kết quả cho thấy cách tiếp cận của chúng tôi có quy mô hiệu quả và nhanh hơn đáng kể so với thời gian chạy hàm mũ e^t của phương pháp BiRRT (ví dụ: chính sách của chúng tôi giải quyết nhiệm vụ nhanh hơn 15 lần trong 10 nhiệm vụ nhánh).

Liên hệ thời gian tính toán mô hình với số lượng tác tử



Hình 5.2. Kết quả so sánh giữa phương pháp đề tài xây dựng và hai phương pháp cơ sở - BiRRT và sao chép hành động.

5.3.2. Tỷ lệ thành công khi tăng số lượng tác tử.

Tính linh hoạt trong quy mô nhóm. Thứ hai, để đánh giá tính linh hoạt của chính sách đối với quy mô nhóm, chúng tôi trực tiếp kiểm tra chính sách của mình trên thang đánh giá (chỉ được đào tạo về nhiệm vụ 1-4 cánh tay) trên nhiệm vụ 5-10 cánh tay (Bảng 1).

Trung bình, chính sách của chúng tôi đạt được > 90% tỷ lệ thành công cho các nhiệm vụ lập kế hoạch chuyển động từ 1-10 cánh tay. Do tính chất phi tập trung của nó, mô hình lập kế hoạch chuyển động của chúng tôi vẫn giữ được tỷ lệ thành công gần như tương tự khi có nhiều nhánh hơn được thêm vào hệ thống, miễn là độ khó của nhiệm vụ vẫn giữ nguyên.

Tuy nhiên, tỷ lệ thành công có xu hướng giảm dần khi quy mô nhóm tăng lên là do thước đo thành công của chúng tôi, kết hợp với mỗi nhóm có xác suất thành công xấp xỉ không đổi cho từng mức độ khó (ví dụ: nếu tỷ lệ thành công của một nhóm là 0,98 thì tỷ lệ thành công cho quy mô nhóm là N trên nhiệm vụ dễ dàng là khoảng 0,98 N).

Bảng 1. Tỷ lệ thành công của quá trình thực hiện trên chế độ tĩnh.

Mức	Dễ	Vừa	Khó	Trung bình
1	0.986	-	-	0.986

2	0.958	0.904	0.876	0.943
3	0.969	0.945	0.915	0.960
4	0.956	0.937	0.937	0.951
5	0.948	0.935	0.914	0.943
6	0.947	0.915	0.900	0.938
7	0.924	0.901	0.894	0.918
8	0.909	0.907	0.920	0.910
9	0.912	0.928	0.864	0.910
10	0.908	9.04	0.881	0.905

5.3.3. Hiệu quả hoạt động của mô hình huấn luyện.

Trong trường hợp xét tính hợp tác của hệ đa robot thông qua phần thưởng nhóm. Thuật toán của chúng tôi đạt được sự hợp tác nhiều bên với nhóm phùng và phối hợp bằng cách cho phép cánh tay robot quan sát trạng thái của cánh tay khác. Để xác nhận hiệu quả hợp tác của chính sách xây dựng, chúng tôi so sánh tác tử của mình với một số phương pháp: Không có mô phạm đầu vào, Không dùng học tăng cường, Chính sách [Selfish] với một chiến lược cách cánh tay không quan tâm tới hoạt động cánh tay khác, chỉ tập trung lấy điểm thưởng của mình thưởng +0,1 bất cứ khi nào một cánh tay cá nhân đạt được mục tiêu và bị phạt -0,05 bất cứ khi nào nó va chạm mà không có phần thưởng đặc biệt của cả hệ khi tất cả các cánh tay đều đạt được tư thế mục tiêu. Dưới đây, Bảng 2 biểu diễn hiệu quả của hoạt động trên các models khác nhau.

Bảng 2. So sánh hiệu quả hợp tác giữa các phương pháp

Số robot	1	2	3	4
Không có đào tạo mẫu	0.001	0.000	0.000	0.000
Không dung học tăng cường	0.403	0.685	0.646	0.852
Chính sách Selfish	0.139	0.001	0.000	0.000
Chính sách cá nhân	0.224	0.023	0.000	0.000
Tác tử đề tài xây dựng	0.988	0.943	0.960	0.951

Để xác thực tính hợp tác của việc cho phép các cánh tay quan sát trạng thái cánh tay khác, chúng tôi so sánh chính sách của mình với đường cơ sở [Chính sách cá nhân], được huấn luyện từ đầu mà không quan sát các cánh tay khác. Vì các cánh tay không quan sát nhau nên những nỗ lực tiếp cận khi có sự hiện diện của các cánh tay khác có thể dẫn đến hình phạt va chạm và do đó có hiệu suất quay trở lại thấp hơn so với khi đứng yên. Bảng 2 cho thấy chính

sách của chúng tôi vượt trội so với đường cơ sở chủ nghĩa cá nhân, chứng tỏ rằng việc quan sát các nhóm khác cho phép phối hợp hiệu quả.

Với trường hợp không có dữ liệu mô phạm hành động mẫu, chính sách hiếm khi nhận được phản hồi tích cực và thường xuyên phải nhận các điểm phạt do xảy ra va chạm. Do đó, tác nhân học cách tránh va chạm bằng cách đứng yên, dẫn đến tỷ lệ thành công là 0% (Bảng 2).

Ngoài ra, chính sách sao chép hành vi điều khiển phân tán [không học tăng cường] giúp tối đa hóa khả năng ghi lại các hành động của dữ liệu mẫu dựa trên các quan sát và bắt chước khi gặp tính huống tương tự cũng hoạt động kém (Bảng 2). Tuy có dữ liệu điều khiển mẫu nhưng trường hợp xảy ra trùng khớp hoàn toàn tất cả vị trí của góc khớp các robot với mẫu rất hiếm xảy ra, nên mô hình này không phát huy được hiệu quả của nó. Chính sách của chúng tôi cũng hoàn thành nhiệm vụ nhanh hơn chính sách sao chép hành vi, cho thấy rằng chính sách của chúng tôi cải thiện tính tối ưu thông qua việc tự khám phá ngoài dữ liệu mẫu mà nó nhận được.

5.4. Kết luận.

Đề tài nghiên cứu xây dựng trình bày một mô hình lập kế hoạch chuyển động cho hệ cánh tay robot với phương pháp điều khiển khép kín có khả năng mở rộng và linh hoạt phù hợp với quy mô nhóm. Các hệ thống rô-bốt đa cánh tay truyền thống dựa vào các mô hình lập kế hoạch chuyển động tập trung, có thời gian chạy thường tăng theo cấp số nhân theo hàm mũ e^x khi tăng số robot tham gia vào hệ, do đó, không thể xử lý các môi trường động bằng điều khiển vòng lặp mở. Trong nghiên cứu này, chúng tôi đã giải quyết vấn đề này bằng phương pháp học tăng cường đa tác tử, trong đó một chính sách phi tập trung được huấn luyện để điều khiển một cánh tay robot trong hệ thống nhiều cánh tay nhằm đạt được tư thế tác nhân cuối cùng dựa trên các quan sát về trạng thái không gian làm việc và mục tiêu cuối cùng của nó. tư thế tác động. Chính sách này được đào tạo bằng cách sử dụng Soft Actor-Critic với sự minh họa của chuyên gia về thuật toán lập kế hoạch chuyển động dựa trên lấy mẫu (tức là BiRRT).

Bằng cách tận dụng các thuật toán lập kế hoạch cổ điển, chúng ta có thể cải thiện hiệu quả học tập của thuật toán học tăng cường trong khi vẫn giữ được thời gian tính toán nhanh của mạng nơ-ron. Chính sách cho kết quả có thời gian chạy tăng tuyến tính và có thể được triển khai trên các hệ thống đa cánh tay khi thay đổi quy mô hệ thống. Nhờ phương pháp điều khiển phân tán vòng kín, phương pháp tiếp cận của chúng tôi khái quát hóa cho 5-10 hệ thống nhiều nhánh và mục tiêu di chuyển động (>90% tỷ lệ thành công cho hệ thống 10 nhánh), mặc dù chỉ được đào tạo về nhiệm vụ lập kế hoạch 1-4 nhánh với mục tiêu tĩnh.

TÀI LIỆU THAM KHẢO

¹ B. K. Kim and K. G. Shin, "Minimum-time path planning for robot arms and their dynamics," IEEE Transactions on Systems, Man, and Cybernetics, vol. SMC-15, no. 2, pp. 213-223, March/April 1985.

-
- ² H. Choset et al., *Principles of Robot Motion: Theory, Algorithms, and Implementation*. Cambridge: The Massachusetts Institute of Technology Press, 2005.
- ³ S. M. LaValle. *Rapidly-exploring random trees: A new tool for path planning*. 1998.
- ⁴ J. Kurosu, A. Yorozu, and M. Takahashi. Simultaneous dual-arm motion planning for minimizing operation time. *Applied Sciences*, 2017.
- ⁵ J. P. van den Berg and M. H. Overmars. Prioritized motion planning for multiple robots. In *2005 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 430–435, 2005.
- ⁶ S. S. Mirrazavi Salehian, N. Figueroa, and A. Billard. A unified framework for coordinated multi-arm motion planning. *The International Journal of Robotics Research*, 37(10):1205–1232, 2018.
- ⁷ K. Solovey, O. Salzman, and D. Halperin. Finding a needle in an exponential haystack: Discrete RRT for exploration of implicit roadmaps in multi-robot motion planning. *CoRR*, abs/1305.2889, 2013.
- ⁸ O. Salzman, K. Solovey, and D. Halperin. Motion planning for multilink robots by implicit configuration-space tiling. *IEEE Robotics and Automation Letters*, 1(2):760–767, 2016.
- ⁹ S. M. LaValle. *Planning algorithms*. Cambridge university press, 2006.
- ¹⁰ S. Karaman and E. Frazzoli. Sampling-based algorithms for optimal motion planning. *The international journal of robotics research*, 30(7):846–894, 2011.
- ¹¹ A. H. Qureshi and Y. Ayaz. Potential functions based sampling heuristic for optimal path planning. *Autonomous Robots*, 40(6):1079–1093, 2016.
- ¹² Z. Tahir, A. H. Qureshi, Y. Ayaz, and R. Nawaz. Potentially guided bidirectionalized rrt* for fast optimal path planning in cluttered environments. *Robotics and Autonomous Systems*, 108:13–27, 2018.
- ¹³ A. H. Qureshi and Y. Ayaz. Intelligent bidirectional rapidly-exploring random trees for optimal motion planning in complex cluttered environments. *Robotics and Autonomous Systems*, 68: 1–11, 2015.
- ¹⁴ L. E. Kavraki, P. Svestka, J. . Latombe, and M. H. Overmars. Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE Transactions on Robotics and Automation*, 12(4):566–580, 1996.
- ¹⁵ B. Ichter, J. Harrison, and M. Pavone. Learning sampling distributions for robot motion planning. *CoRR*, abs/1709.05448, 2017.
- ¹⁶ A. H. Qureshi and M. C. Yip. Deeply informed neural sampling for robot motion planning. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 6582–6588. IEEE, 2018.
- ¹⁷ N. Jetchev and M. Toussaint. Fast motion planning from experience: trajectory prediction for speeding up movement generation. *Autonomous Robots*, 34(1-2):111–127, 2013.
- ¹⁸ A. H. Qureshi, A. Simeonov, M. J. Bency, and M. C. Yip. Motion planning networks. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 2118–2124. IEEE, 2019.
- ¹⁹ D. Berenson, P. Abbeel, and K. Goldberg. A robot path planning framework that learns from experience. In *2012 IEEE International Conference on Robotics and Automation*, pages 3671–3678. IEEE, 2012.
- ²⁰ R. Shome, K. Solovey, A. Dobson, D. Halperin, and K. E. Bekris. drrt*: Scalable and informed asymptotically-optimal multi-robot motion planning. *CoRR*, abs/1903.00994, 2019.
- ²¹ H. Zhang, E. Heiden, S. Nikolaidis, J. J. Lim, and G. S. Sukhatme. Auto-conditioned recurrent mixture density networks for learning generalizable robot skills. *arXiv preprint arXiv:1810.00146*, 2018.

-
- ²² M. Pfeiffer, M. Schaeuble, J. Nieto, R. Siegwart, and C. Cadena. From perception to decision: A data-driven approach to end-to-end motion planning for autonomous ground robots. In 2017 IEEE International Conference on Robotics and Automation (ICRA), pages 1527–1533. IEEE, 2017.
- ²³ M. J. Bency. Towards Neural Network Embeddings of Optimal Motion Planners. PhD thesis, UC San Diego, 2018.
- ²⁴ A. Faust, K. Oslund, O. Ramirez, A. Francis, L. Tapia, M. Fiser, and J. Davidson. Prm-rl: Long-range robotic navigation tasks by combining reinforcement learning and sampling-based planning. In 2018 IEEE International Conference on Robotics and Automation (ICRA), pages 5113–5120. IEEE, 2018.
- ²⁵ A. Tamar, Y. Wu, G. Thomas, S. Levine, and P. Abbeel. Value iteration networks. In Advances in Neural Information Processing Systems, pages 2154–2162, 2016.
- ²⁶ T. Jurgenson and A. Tamar. Harnessing reinforcement learning for neural motion planning. CoRR, abs/1906.00214, 2019.
- ²⁷ T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. arXiv preprint arXiv:1509.02971, 2015.
- ²⁸ R. Shome and K. E. Bekris. Anytime multi-arm task and motion planning for pick-and-place of individual objects via handoffs. In 2019 International Symposium on Multi-Robot and Multi-Agent Systems (MRS), pages 37–43. IEEE, 2019.
- ²⁹ R. Shome, K. Solovey, J. Yu, K. E. Bekris, and D. Halperin. Fast, high-quality dual-arm rearrangement in synchronous, monotone tabletop setups. CoRR, 2018.
- ³⁰ J. P. van den Berg and M. H. Overmars. Roadmap-based motion planning in dynamic environments. IEEE Transactions on Robotics, 21(5):885–897, 2005.
- ³¹ J. Alonso-Mora, A. Breitenmoser, M. Rufli, P. Beardsley, and R. Siegwart. Optimal reciprocal collision avoidance for multiple non-holonomic robots. In Distributed autonomous robotic systems, pages 203–216. Springer, 2013.
- ³² M. Everett, Y. F. Chen, and J. P. How. Motion planning among dynamic, decision-making agents with deep reinforcement learning. CoRR, abs/1805.01956, 2018. URL <http://arxiv.org/abs/1805.01956>.
- ³³ S. H. Semnani, H. Liu, M. Everett, A. de Ruiter, and J. P. How. Multi-agent motion planning for dense and dynamic environments via deep reinforcement learning. IEEE Robotics and Automation Letters, 5(2), 2020.
- ³⁴ S. Hochreiter and J. Schmidhuber. Long short-term memory. Neural computation, 9(8):1735–1780, 1997.
- ³⁵ Stefano V. Albrecht, Filippos Christianos, Lukas Schäfer. Multi-Agent Reinforcement Learning: Foundations and Modern Approaches. MIT Press, 2024. <https://www.marl-book.com/>
- ³⁶ Lowe, Ryan; Wu, Yi (2020). "Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments". arXiv:1706.02275v4
- ³⁷ Baker, Bowen (2020). "Emergent Reciprocity and Team Formation from Randomized Uncertain Social Preferences". NeurIPS 2020 proceedings. arXiv:2011.05373
- ³⁸ Hughes, Edward; Leibo, Joel Z.; et al. (2018). "Inequity aversion improves cooperation in intertemporal social dilemmas". NeurIPS 2018 proceedings. arXiv:1803.08884
- ³⁹ Jaques, Natasha; Lazaridou, Angeliki; Hughes, Edward; et al. (2019). "Social Influence as Intrinsic Motivation for Multi-Agent Deep Reinforcement Learning". Proceedings of the 35th International Conference on Machine Learning. arXiv:1810.08647
- ⁴⁰ E. Coumans and Y. Bai. Pybullet, a python module for physics simulation in robotics, games and machine learning, 2017.